
PoMtVRS: Preference-Optimized Multi-Task Vehicle Routing Solver with Preference Gating

Dian Meng^{1,2} Yaoxin Wu³ Yaqing Hou^{1,2,*} Zhiguang Cao⁴

Abstract

Multi-task vehicle routing solvers via deep reinforcement learning have attracted broad attention and achieved significant progress in handling multiple constraints. However, existing neural solvers still face critical challenges, including insufficient representation, unstable training, and inefficient exploration in large combinatorial action spaces, which often prevents performance from meeting its full potential. To address these issues, we propose PoMtVRS (Preference-Optimized Multi-Task Vehicle Routing Solver with Preference Gating), a plug-and-play framework that jointly improves decoder representations and exploration efficiency through a synergistic combination of decoder-side augmentation and preference-driven optimization. Specifically, we introduce the preference optimization objective to learn relative comparisons among candidate solutions for different routing tasks, encouraging a higher generation probability of better solutions. Meanwhile, we design a preference-gated block that adaptively modulates decoder representations via sparse gated attention and nonlinear residual refinement. Extensive experiments demonstrate that PoMtVRS elevates state-of-the-art unified neural VRP backbones, achieving leading performance in multi-task benchmarks and stronger generalization. The code is available at: <https://github.com/Regina921/PoMtVRS>.

1. Introduction

Vehicle Routing Problems (VRPs) are a class of classic combinatorial optimization problems (COPs) in operations research and computer science, widely recognized as NP-hard, with broad applications in transportation services (Ge et al., 2019), logistics (Konstantakopoulos et al., 2022), and intelligent manufacturing (Zhang et al., 2023). Traditional solvers mainly fall into exact methods and handcrafted heuristics. Exact algorithms (e.g., branch-and-bound/branch-and-cut) typically incur exponential computational cost as the problem size grows, which makes large-scale deployment expensive (Laporte & Nobert, 1983). Heuristics can produce near-optimal solutions within reasonable time, yet often rely heavily on domain expertise and extensive iterative search, which limits portability and efficiency under frequent constraint switching (Helsgaun, 2017; Vidal et al., 2012).

Recently, Neural Combinatorial Optimization (NCO) has emerged as a promising paradigm for VRPs (Bengio et al., 2021). By learning constructive or improvement-based neural heuristics, NCO solvers can generate solutions autoregressively and optimize policies end-to-end with trajectory sampling (e.g., REINFORCE (Williams, 1992)), achieving competitive solution quality and inference speed on specific VRP variants (Kwon et al., 2020; Drakulic et al., 2023; Luo et al., 2023; Meng et al., 2025a; Luo et al., 2025; Wu et al., 2026), together with some out-of-distribution generalization (Bi et al., 2022; Hou et al., 2023; Zheng et al., 2024; Zhou et al., 2023b). However, practical applications involve diverse node attributes and solution constraints. Under such variability, specialized neural solvers often require training separate models or extensive retuning for each variant, which leads to high maintenance cost and limited scalability.

Consequently, the community has begun to move toward unified multi-task VRP solvers that exploit shared structure across variants for knowledge transfer and combinatorial generalization, e.g., modeling variants as compositions of constraint attributes (Liu et al., 2024; Zhou et al., 2024), improving unified environments (Berto et al., 2025), and enhancing unified solvers through better architecture design, dynamic context understanding, and training recipes (Li et al., 2025; Pan et al., 2025b; Liu et al., 2025; Gui et al., 2026). Despite strong progress, two bottlenecks remain.

¹School of Computer Science and Technology, Dalian University of Technology (DUT) ²Key Laboratory of Social Computing and Cognitive Intelligence (DUT), Ministry of Education, China ³Department of Industrial Engineering and Innovation Sciences, Eindhoven University of Technology ⁴School of Computing and Information Systems, Singapore Management University. Correspondence to: Yaqing Hou <houyq@dlut.edu.cn>.

First, optimal decision rules can change drastically across constraint switches, while standard decoders largely rely on a fixed attention context, making it difficult to perform fine-grained, state-conditional modulation during decoding and often resulting in limited expressiveness and unstable training. Second, most neural solvers rely on reinforcement learning (RL) with absolute reward values, where reward scales vary across variants, and return differences shrink as policies improve, weakening the learning signal. Combined with cross-task gradient interference, this causes high variance, inefficient exploration, and slow convergence.

To address these issues, we propose PoMtVRS (Preference-Optimized Multi-Task Vehicle Routing Solver with Preference Gating), a plug-and-play framework that jointly improves decoder expressiveness and exploration efficiency via architectural augmentation and preference-driven optimization. First, to mitigate attention sink and unstable training under constraint switching, we insert a preference-gated block (PGB) after the multi-head attention (MHA) output. The block combines head-wise gating, state mapping, and nonlinear residual refinement, enabling constraint-conditioned modulation of attention pathways and contextual features for more robust decoding representations. Second, to overcome reward-scale mismatch and diminishing learning signals, we introduce preference optimization (PO) to learn relative preferences from pairwise comparisons between candidate solutions of the same instance. Based on the Bradley-Terry (BT) model of log-likelihood differences, we transform quantitative reward signals into qualitative preference signals, effectively improving exploration efficiency and solution quality. Finally, PoMtVRS can be seamlessly integrated into existing neural solvers and has demonstrated its efficiency and practicality on two representative neural methods. It achieved state-of-the-art performance on multi-task VRPs and exhibits excellent generalization capabilities across different problem scales. Our contributions are summarized as follows:

- We propose PoMtVRS, a plug-and-play preference-learning framework for multi-task VRPs, which jointly enhances representation capacity and exploration efficiency through cooperative decoder augmentation and preference-driven optimization.
- We design a preference-gated block that adaptively reorganizes decoder representations via head-wise gated attention modulation and nonlinear residual refinement. Coupled with the PO objective, we model pairwise preferences with the BT likelihood so that the two work in synergy to enable more stable optimization and more efficient exploration of the solution space.
- Extensive experiments show that PoMtVRS can be seamlessly integrated into state-of-the-art neural back-

bone methods, achieving heading performance and exhibiting strong generalization capability.

2. Related Works

2.1. Neural combinatorial optimization for VRPs

NCO has become a prominent learning paradigm for solving VRPs, where deep policies are trained as generalizable heuristics. Existing methods are commonly categorized into construction-based solvers that generate solutions autoregressively and improvement-based solvers that iteratively refine an initial solution. Early work models routing as sequential decision making and trains autoregressive policies end-to-end with REINFORCE, enabling direct instance-to-route generation (Vinyals et al., 2015; Bello et al., 2017; Nazari et al., 2018). The Transformer-based attention model (AM) substantially improves solution quality and scalability and has become a dominant backbone for single-task neural solvers (Kool et al., 2019), while POMO further boosts performance by exploiting solution symmetries with multi-start sampling (Kwon et al., 2020). Subsequent studies enhance constructive frameworks and explore stronger inductive representations (Kwon et al., 2021; Li et al., 2021; Kim et al., 2022; Berto et al., 2023; Chen et al., 2023; Chalumeau et al., 2023; Drakulic et al., 2023; Luo et al., 2023; Hottung et al., 2024; Meng et al., 2025b). In parallel, improvement-based methods combine learned local operators with large-neighborhood search to achieve better solutions under larger inference budgets (Chen & Tian, 2019; Lu et al., 2020; Hottung & Tierney, 2020; d O Costa et al., 2020; Wu et al., 2021; Xin et al., 2021; Hudson et al., 2022; Zhou et al., 2023a; Ma et al., 2023). Nevertheless, most neural solvers are still trained and tuned for a single variant under a fixed distribution, which can be brittle under constraint switching or unified multi-variant settings.

2.2. Multi-Task Learning for VRPs

Learning unified VRP solvers by exploiting shared structures across variants has gained increasing attention. Wang & Yu (2025) propose a multi-armed bandit scheme for task scheduling and sampling allocation to improve training efficiency under a fixed budget. Lin et al. (2024) pretrain a backbone on a canonical VRP and adapt it to target variants via low-dimensional modules, enabling parameter-efficient transfer. Drakulic et al. (2025) trains a generalist CO agent with supervised learning to promote cross-task reuse, while Jiang et al. (2024) leverages LLMs to map instance descriptions into a unified embedding space for representation sharing. For unified solvers, Liu et al. (2024) model variants as compositions of primitive constraint attributes for zero-shot generalization, and Zhou et al. (2024) improves multi-variant performance with mixture-of-experts. Berto et al. (2025) further enhances training stability and

scalability via modern Transformer components and mixed-batch training. Li et al. (2025) explicitly encodes constraint prompts with a dual-attention design. Beyond multi-task, Goh et al. (2025) study the more realistic multi-task and multi-distribution setting with hierarchical inductive biases, and Pan et al. (2025b) reuse basic solvers through plug-and-play expert adapters with gated mixtures to mitigate model growth. Despite substantial progress, most unified solvers still rely on RL with absolute reward values, making training sensitive to diminishing learning signals and gradient interference, which degrades stability and sample efficiency.

2.3. Preference Optimization

PO has attracted increasing attention because it can align models with human preferences without explicitly training a reward model as required by conventional Reinforcement Learning from Human Feedback (RLHF). RLHF typically learns a reward model from ranked feedback and then fine-tunes the policy via RL (Stiennon et al., 2020; Ouyang et al., 2022). To simplify this pipeline, Rafailov et al. (2023) proposed Direct Preference Optimization (DPO), which bypasses reward modeling and directly optimizes the likelihood using preference pairs. This line of work has since been extended to improve preference modeling and objective stability, including interpreting DPO as implicit reward modeling (Dong et al., 2023), generalizing pairwise supervision to ranking-based feedback (Song et al., 2024), and developing simpler losses (Xu et al., 2023) and reference-free variants such as SimPO (Meng et al., 2024). A survey provides a more comprehensive overview of these advances (Xiao et al., 2024). PO has also been increasingly adopted in NCO. Pan et al. (2025a) introduced PO for COPs, and Liao et al. (2025) further improved training stability and effectiveness by proposing COP-tailored preference construction and objective-guided losses. Fan et al. (2026) proposed POCCO, extending PO to multi-objective COPs. However, most existing studies focus on single-problem settings, and systematic investigation of PO for VRPs, especially under multi-task solver regimes, remains limited.

3. Preliminaries

3.1. Problem Definition

A VRP instance of size N on a complete graph $\mathcal{G} = \{V, E\}$, where $V = \{v_0, v_1, \dots, v_N\}$ ($|V| = N + 1$) consists of a depot v_0 and N customers $\{v_1, \dots, v_N\}$. Each node $v_i \in V$ has 2D coordinates (x_i, y_i) , and each customer additionally carries constraint attributes c_i . The edge set is $E = \{e(v_i, v_j) \mid 0 \leq i \neq j \leq N\}$, and each edge induces a travel cost e_{ij} . Vehicles are homogeneous with capacity Q . A solution τ is a set of sub-routes. Each sub-route starts from the depot, visits a subset of customers, and satisfies feasibility: every customer is served exactly once and all

constraints hold throughout execution. We minimize the total Euclidean length $c(\tau)$: $\tau^* = \arg \min_{\tau} c(\tau)$, i.e., the minimum-cost feasible routing plan. Taking the capacitated vehicle routing problem (CVRP) as the base variant, combining the capacity constraint (C) with any subset of the four additional constraints (O)/(B)/(L)/(TW) yields 16 VRP variants. **Capacity constraint (C)**: Each customer v_i has a demand q_i , and the vehicle load must satisfy $q \leq Q$. **Open Routes (O)**: A vehicle is not required to return to the depot after finishing its assigned customers. **Backhauls (B)**: Linehaul customers satisfy demands $b_i > 0$, while backhaul customers satisfy $b_i < 0$. A sub-route may include both types, but all linehaul customers must be visited before any backhaul customer. **Duration Limit (L)**: Each sub-route must satisfy an upper bound on its total travel duration (or length). **Time Window (TW)**: Each customer v_i has a service time window $[e_i, f_i]$ and a service duration s_i . The vehicle must arrive no later than f_i . If it arrives early, it must wait until e_i starting service, ensuring the service start time lies within $[e_i, f_i]$ (see Appendix A for detailed definitions).

3.2. Transformer Decoder

Most neural VRP solvers follow a Transformer architecture, where the encoder maps an instance graph to joint node representations $\mathbf{H} = [\mathbf{h}_0, \mathbf{h}_1, \dots, \mathbf{h}_N]^\top \in \mathbb{R}^{(N+1) \times d}$ with model dimension d . At the decoding step t , a context vector $h_t \in \mathbb{R}^d$ is formed from the current partial solution and is used to produce the probability of the next node.

QKV Linear Projections. Let H is the number of attention heads and $d_h = d/H$ is the per-head dimension. For each head $h \in \{1, \dots, H\}$, the decoder projects the context and node embeddings to queries, keys, and values:

$$q_t^{(h)} = h_t W_Q^{(h)}, K^{(h)} = \mathbf{H} W_K^{(h)}, V^{(h)} = \mathbf{H} W_V^{(h)}, \quad (1)$$

where $W_Q^{(h)}, W_K^{(h)}, W_V^{(h)}$ are learnable parameters.

Scaled Dot-Product Attention (SDPA). Each head computes compatibility scores via scaled dot products and applies a feasibility mask, followed by a softmax normalization. The head output $z_t^{(h)}$ is the weighted sum of values:

$$z_t^{(h)} = \text{softmax} \left(\frac{q_t^{(h)} K^{(h)\top}}{\sqrt{d_h}} \right) V^{(h)}, \quad (2)$$

where $z_t^{(h)} \in \mathbb{R}^{1 \times d_h}$, and $\text{softmax}(\cdot)$ ensures the attention weights are non-negative and sum to 1 across each row.

Multi-Head Concatenation. The H head outputs are concatenated along the feature dimension:

$$M_t^H = \text{Concat}(z_t^{(1)}, \dots, z_t^{(H)}). \quad (3)$$

The SDPA output is fused by the output projection W_o :

$$A_t = W_o M_t^H, \quad (4)$$

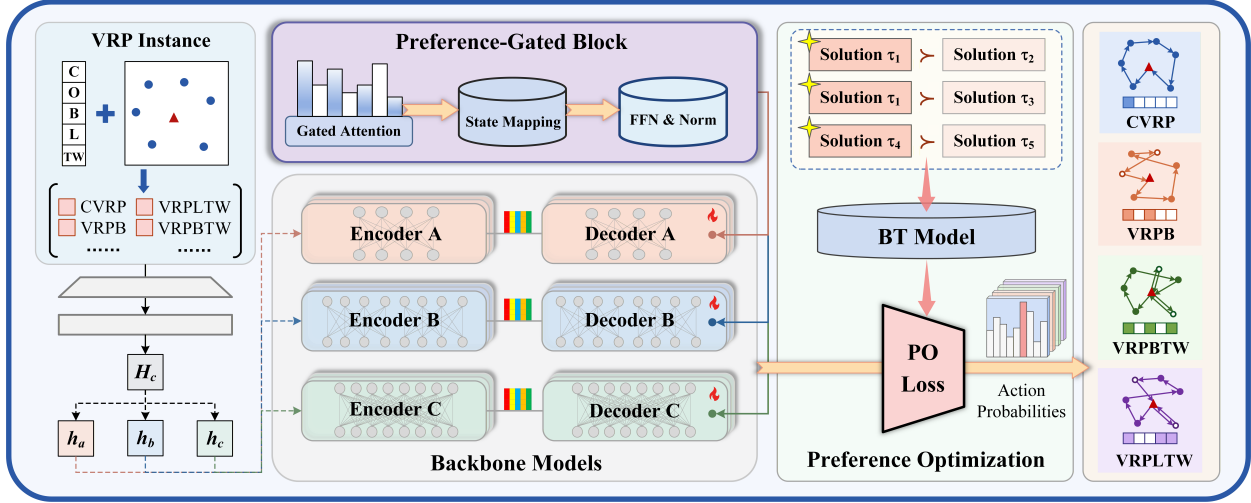


Figure 1. An overview of the proposed PoMtVRS. It integrates seamlessly into typical encoder-decoder frameworks: Under the unified VRP environment, PoMtVRS augments standard neural solvers by inserting a Preference-Gated Block (PGB) into the decoder, which enhances decoder representation and adaptively refines attention context. Training adopts preference optimization (PO), which replaces scalarized rewards by transforming intra-instance pairwise preferences into a Bradley-Terry (BT) likelihood and PO loss, yielding a stable reward signal to optimize the PL policy.

Compatibility and Softmax. Given the final vector A_t , we compute a score γ_t using a masked single-head attention:

$$\gamma_t = \begin{cases} C \cdot \tanh\left(\frac{A_t K^{(h)}}{\sqrt{d_h}}\right), & \text{if unvisited,} \\ -\infty, & \text{otherwise,} \end{cases} \quad (5)$$

where C is a scaling constant. The final selection probability ρ_t for the step t is computed via the softmax function:

$$\rho_t = \text{softmax}(\gamma_t), \quad (6)$$

At each decoding step, an action τ_t is sampled according to ρ_t . Repeating this process for n steps yields the full solution.

3.3. Reinforcement Learning for Solving VRPs

A common NCO paradigm models VRP construction as a Markov decision process (MDP) and uses an attention-based policy p_θ (Kool et al., 2019). For instance $\mathcal{G}$, the decoder selects a feasible action τ_t at each step t given state s_t (defined by $\mathcal{G}$ and the partial solution). A complete solution is $\tau = (\tau_1, \dots, \tau_T)$ with probability factorized as:

$$p_\theta(\tau | \mathcal{G}) = \prod_{t=1}^T p_\theta(\tau_t | s_t, \mathcal{G}), \quad (7)$$

where $p_\theta(\tau_t | s_t, \mathcal{G})$ is the conditional probability of choosing action τ_t at state s_t . After constructing τ , the environment returns a scalar reward $R(\tau, \mathcal{G})$. Over an instance distribution $P(\mathcal{G})$, policy gradient training maximizes the expected reward as follows:

$$\mathcal{L}_{\text{RL}}(\theta) = \mathbb{E}_{\mathcal{G} \sim P(\mathcal{G})} \mathbb{E}_{\tau \sim p_\theta(\cdot | \mathcal{G})} [R(\tau, \mathcal{G})]. \quad (8)$$

The policy network is typically optimized using the REINFORCE, which maximizes the expected reward $\mathcal{L}_{\text{RL}}(\theta)$ via the following gradient estimator:

$$\nabla_\theta \mathcal{L}_{\text{RL}}(\theta) = \mathbb{E}_{p_\theta(\tau | \mathcal{G})} \left[(R(\tau, \mathcal{G}) - b(\mathcal{G})) \nabla_\theta \log p_\theta(\tau | \mathcal{G}) \right], \quad (9)$$

where $b(\mathcal{G})$ is a baseline that depends only on the instance $\mathcal{G}$ and reduces gradient variance. Despite its simplicity and lack of optimal supervision, REINFORCE depends on absolute rewards $R(\tau, \mathcal{G})$. In multi-task VRPs, variant-dependent reward scales and noise often induce high-variance gradients and baseline sensitivity, while shrinking within-instance reward gaps weaken the learning signal as training progresses. To address this, we replace absolute rewards with relative preferences via PO, yielding more stable training for multi-task VRP solvers.

4. Method

4.1. Overview

We propose PoMtVRS, a preference-driven framework in a unified VRP environment. Following the standard Transformer paradigm, an encoder produces node embeddings and attention caches, and an autoregressive decoder constructs feasible routes via masked decoding. PoMtVRS targets two major issues: (i) constraint switches alter decision rules, while fixed attention context limits state-conditional modulation and destabilizes training. (ii) RL with scalarized rewards suffers from reward scale mismatch and shrinking return gaps, amplifying gradient interference and hurting exploration. To address them, PoMtVRS inserts a lightweight

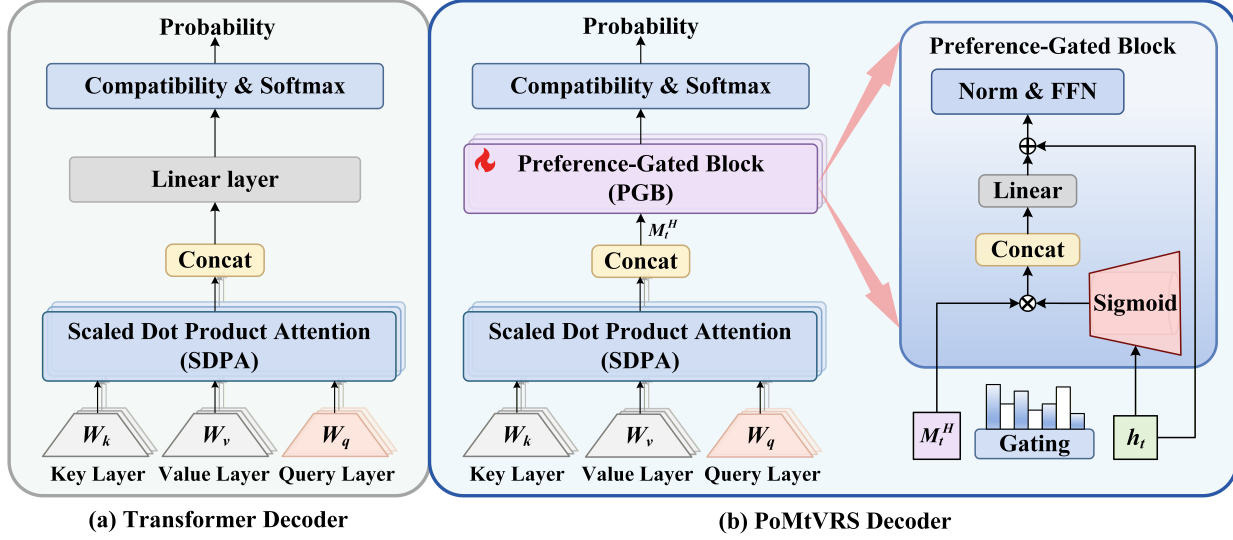


Figure 2. Decoder comparison between the backbone and PoMtVRS. (a) The standard decoder obtains a context representation via Scaled Dot Product Attention (SDPA) and directly applies a linear compatibility scoring layer followed by Softmax to produce action probabilities. (b) PoMtVRS inserts the Preference-Gated Block (PGB) between the attention output and the scoring layer to perform state-conditioned reorganization of the context vector. PGB combines head-wise gating, state mapping and nonlinear residual refinement using normalization and feed-forward. The refined context is then fed into the compatibility layer and softmax to compute probabilities.

preference-gated block and replaces RL with PO.

The overview of PoMtVRS is illustrated in Figure 1. For each instance, we perform multi-start rollouts to obtain candidate solutions and their trajectory log-likelihoods. We insert a plug-and-play preference-gated block between the MHA output and the compatibility layer to refine decoder representations. The block applies head-wise gating, attribute mapping, and nonlinear residual refinement, which improves robustness under diverse constraints (Section 4.2).

For training, PO converts raw rewards into within-instance pairwise preferences, so the policy assigns higher probability to higher-quality solutions. Because it depends on relative ordering rather than absolute values, PO mitigates reward-scale mismatch across tasks and provides denser supervision within each instance, which improves optimization stability and sample efficiency (Section 4.3). Overall, PoMtVRS combines decoder-side preference modulation with preference-driven optimization to form a stable, plug-and-play framework for neural solvers.

4.2. Preference-Gated Block (PGB)

Neural solvers typically use a Transformer decoder. At each step, a query from the partial solution attends to node keys/values to form a context vector, which is mapped to the next-node probability. In multi-task VRPs, constraint switches can alter the optimal policy, so attention context alone may miss heterogeneous signals. We thus introduce a lightweight preference-gated block for task-adaptive modulation, inserted between the MHA output M_t^H (Section 3.2)

and the compatibility. The block has three components.

Head-wise Gating. Inspired by gated attention (Qiu et al., 2026), we introduce head-wise gating in the decoder. Instead of gating SDPA outputs, we apply the gating mechanism after multi-head concatenation, preserving the original MHA and enabling plug-and-play modulation. Specifically, we reshape M_t^H by heads and compute a sigmoid gate $g_t^{(h)}$ from h_t , which modulates each head independently:

$$\tilde{y}_t^{(h)} = \mathcal{F}(M_t^H) \odot \sigma(h_t W_g), \quad (10)$$

where σ is an activation function (e.g., sigmoid), W_g refers to the learnable parameters of gate, $\mathcal{F}(\cdot)$ is an inverse-concatenation operator, and $\tilde{y}_t^{(h)}$ is the gated output. The gated multi-head output is then projected to the model space to obtain $y_t \in \mathbb{R}^d$:

$$y_t = \text{Concat}(\tilde{y}_t^{(1)}, \dots, \tilde{y}_t^{(H)}) W_o. \quad (11)$$

State Attribute Mapping. To explicitly inject constraint-related signals, we map state attribute s_t into the embedding space and fuse it with the current node embedding c_t , forming a state-enhanced residual branch:

$$a_t = c_t + W_a s_t, \quad (12)$$

where W_a is learnable parameters of s_t , s_t denotes the dynamic state determined by $\mathcal{G}$ and partial solution $\tau_{<t}$. The vector a_t acts as a state-conditioned anchor, ensuring that subsequent fusion aligns with context embedding and state attributes.

Nonlinear Residual Refinement. Finally, we fuse y_t with a_t through residual normalization and feed-forward transformation to stabilize training and enhance expressiveness:

$$\tilde{u}_t = \text{Norm}(a_t + y_t), \quad (13)$$

$$u_t = \text{Norm}(\tilde{u}_t + \text{FFN}(\tilde{u}_t)), \quad (14)$$

where $\text{Norm}(\cdot)$ denotes a normalization operator, $\text{FFN}(\cdot)$ can be implemented as a feed-forward or a SigLU/parallel gated MLP, and u_t is the preference-enhanced decoding representation used for node scoring and the next node.

4.3. Preference Optimization for solving VRPs

To mitigate the high-variance gradients and inefficient exploration of REINFORCE in multi-task VRP, we adopt PO, replacing raw scalarized rewards with relative preferences. PO encourages the policy to assign higher probability to better routes by generating multiple candidates per instance, forming preference pairs, and optimizing a Bradley-Terry (BT) likelihood on log-likelihood differences.

Generating preference pairs. Each instance $\mathcal{G}$ produces P candidate routes $\{\tau^{(p)}\}_{p=1}^P$, which P is the number of starts. The policy is $\pi_\theta(\tau \mid \mathcal{G})$ with parameters θ . For a route $\tau^{(p)} = (\tau_1^{(p)}, \dots, \tau_T^{(p)})$, we define log-likelihood as:

$$\ell^{(p)} = \log \pi_\theta(\tau^{(p)} \mid \mathcal{G}) = \sum_{t=1}^T \log \pi_\theta(\tau_t^{(p)} \mid \tau_{<t}^{(p)}, \mathcal{G}), \quad (15)$$

where $\tau_t^{(p)}$ is the action at step t . Let $c(\mathcal{G}, \tau^{(p)})$ be the environment cost (e.g., total tour length). In our setting, The reward is the negative cost: $r^{(p)} = -c(\tau^{(p)}, \mathcal{G})$. For numerical stability, we center rewards within the same instance $\tilde{r}^{(p)} = r^{(p)} - \frac{1}{P} \sum_{q=1}^P r^{(q)}$, which preserves pairwise ordering. We then define pairwise preference labels:

$$y_{i,j} = \mathbb{I}[\tilde{r}^{(i)} > \tilde{r}^{(j)}], \quad (16)$$

where $\mathbb{I}[\cdot]$ is the indicator function. Here, $y_{i,j} \in \{0, 1\}$, $y_{i,j} = 1$ means that route $\tau^{(i)}$ achieves a higher (centered) reward than $\tau^{(j)}$ on the same instance.

Bradley-Terry preference likelihood. The BT model assigns each candidate a scalar preference strength and converts strength differences into win probabilities. We instantiate the strength of route $\tau^{(p)}$ as a scaled log-likelihood:

$$u^{(p)} = \alpha \ell^{(p)}, \quad (17)$$

where $u^{(p)} \in \mathbb{R}$ is the preference strength and $\alpha > 0$ is a temperature/scaling coefficient. The BT probability that route i is preferred over route j is:

$$\Pr(i \succ j \mid \mathcal{G}) = \sigma(u^{(i)} - u^{(j)}) = \sigma(\alpha(\ell^{(i)} - \ell^{(j)})), \quad (18)$$

where $\sigma(\cdot)$ is the sigmoid function. Equivalently, we define the pairwise logit $l_{i,j}$:

$$l_{i,j} = \alpha(\ell^{(i)} - \ell^{(j)}), \quad (19)$$

which directly quantifies the relative preference induced by policy between two candidate routes for the same instance.

Preference optimization objective. We maximize the log-likelihood of all preference comparisons, equivalently minimizing its negative, yielding per-instance PO objective:

$$\mathcal{L}_{\text{PO}}(\theta, \mathcal{G}) = -\frac{1}{P^2} \sum_{i=1}^P \sum_{j=1}^P y_{i,j} \log \sigma(\alpha(\ell^{(i)} - \ell^{(j)})), \quad (20)$$

where P^2 denotes mean normalization over all pairs. Note that $y_{i,i} = 0$ since $\tilde{r}^{(i)} > \tilde{r}^{(i)}$ never holds, so diagonal terms contribute nothing. For batch size B , the training loss is:

$$\mathcal{L}_{\text{PO}}(\theta) = \frac{1}{B} \sum_{b=1}^B \mathcal{L}_{\text{PO}}(\theta, \mathcal{G}_b), \quad (21)$$

where $\mathcal{G}_b$ is the b -th instance. It enforces that if $\tau^{(i)}$ is preferred to $\tau^{(j)}$ (i.e., $y_{i,j} = 1$), the policy assigns higher probability to $\tau^{(i)}$, promoted by enlarging $\ell^{(i)} - \ell^{(j)}$. Compared to REINFORCE updates $\mathbb{E}[(r - b) \nabla_\theta \log p_\theta]$, PO uses relative log-likelihood differences, reducing sensitivity to reward scaling and baseline design.

5. Experiments

Baselines. Traditional Solvers: We benchmark against two classical VRP solvers. PyVRP (Wouda et al., 2024), built on top of HGS-CVRP (Vidal, 2022), serves as a strong heuristic baseline, while Google OR-Tools (Perron & Furnon, 2023) provides a robust industrial reference. Both solvers run on a single CPU core per instance with standard time limits that scale with problem size (e.g., 10s for VRP50 and 20s for VRP100). We parallelize traditional solvers across 16 CPU cores as in (Zhou et al., 2024). **Neural Solvers:** We compare against representative multi-task neural VRP solvers spanning unified training, expert specialization, and constraint-aware architectures. Specifically, we include MTPOMO (Liu et al., 2024), MVMoE (Zhou et al., 2024), RouteFinder (Berto et al., 2025) (RF-POMO, RF-MoE, RF-TE), CaDA (Li et al., 2025) and MoSES (Pan et al., 2025b). For each method, we train and evaluate the models under a unified hardware setup, following the same hyperparameter configurations and problem settings as in RouteFinder (Berto et al., 2025).

PoMtVRS. To validate plug-and-play compatibility, we instantiate PoMtVRS on two representative backbones: (i) MVMoE, a typical POMO-based method for multi-task

PoMtVRS: Preference-Optimized Multi-Task Vehicle Routing Solver with Preference Gating

Table 1. Performance comparison on 1K test instances of 16 VRPs. Best learning-based results are highlighted. The lower, the better ($\downarrow$).

	Solver	N = 50			N = 100				Solver	N = 50			N = 100		
		Obj.	Gap	Time	Obj.	Gap	Time			Obj.	Gap	Time	Obj.	Gap	Time
CVRP	HGS-PyVRP	10.372	*	10.4m	15.628	*	20.8m	VRPTW	HGS-PyVRP	16.031	*	10.4m	25.423	*	20.8m
	OR-Tools	10.572	1.907%	10.4m	16.280	4.178%	20.8m		OR-Tools	16.089	0.347%	10.4m	25.814	1.506%	20.8m
	MTPOMO	10.518	1.408%	2s	15.933	1.986%	8s		MTPOMO	16.409	2.358%	2s	26.410	3.863%	9s
	MVMoE	10.508	1.316%	3s	15.912	1.848%	11s		MVMoE	16.363	2.063%	3s	26.301	3.438%	11s
	RF-POMO	10.508	1.315%	2s	15.908	1.830%	8s		RF-POMO	16.366	2.089%	2s	26.335	3.570%	9s
	RF-MoE	10.499	1.228%	3s	15.877	1.624%	11s		RF-MoE	16.390	2.239%	3s	26.319	3.506%	12s
	RF-TE	10.504	1.276%	2s	15.857	1.507%	8s		RF-TE	16.363	2.069%	2s	26.234	3.177%	8s
	CaDA	10.495	1.186%	3s	15.874	1.601%	13s		CaDA	16.276	1.522%	3s	26.083	2.579%	13s
	MoSES(CaDA)	10.462	0.873%	7s	15.833	1.354%	24s		MoSES(CaDA)	16.262	1.435%	7s	26.032	2.383%	25s
	PoMtVRS(MVMoE)	10.471	0.958%	4s	15.819	1.256%	17s		PoMtVRS(MVMoE)	16.335	1.888%	4s	26.245	3.213%	20s
PoMtVRS(CaDA)	10.459	0.843%	4s	15.768	0.926%	17s	PoMtVRS(CaDA)	16.236	1.272%	4s	25.976	2.151%	20s		
OVRP	HGS-PyVRP	6.507	*	10.4m	9.725	*	20.8m	VRPL	HGS-PyVRP	10.587	*	10.4m	15.766	*	20.8m
	OR-Tools	6.553	0.686%	10.4m	9.995	2.732%	20.8m		OR-Tools	10.570	2.343%	10.4m	16.466	5.302%	20.8m
	MTPOMO	6.718	3.211%	2s	10.210	4.959%	8s		MTPOMO	10.775	1.732%	2s	16.151	2.445%	8s
	MVMoE	6.710	3.086%	3s	10.206	4.921%	11s		MVMoE	10.749	1.499%	3s	16.110	2.186%	11s
	RF-POMO	6.698	2.906%	2s	10.181	4.671%	8s		RF-POMO	10.751	1.525%	2s	16.106	2.166%	8s
	RF-MoE	6.697	2.879%	3s	10.139	4.238%	11s		RF-MoE	10.737	1.388%	3s	16.070	1.937%	11s
	RF-TE	6.684	2.693%	2s	10.121	4.060%	8s		RF-TE	10.748	1.499%	2s	16.051	1.829%	8s
	CaDA	6.670	2.467%	3s	10.127	4.103%	13s		CaDA	10.730	1.330%	3s	16.061	1.875%	13s
	MoSES(CaDA)	6.629	1.857%	7s	10.084	3.679%	24s		MoSES(CaDA)	10.704	1.083%	7s	16.024	1.659%	24s
	PoMtVRS(MVMoE)	6.670	2.474%	4s	10.084	3.674%	17s		PoMtVRS(MVMoE)	10.712	1.151%	4s	16.013	1.571%	17s
PoMtVRS(CaDA)	6.640	2.010%	4s	10.034	3.155%	17s	PoMtVRS(CaDA)	10.698	1.028%	4s	15.951	1.188%	17s		
VRPB	HGS-PyVRP	9.687	*	10.4m	14.377	*	20.8m	OVRPTW	HGS-PyVRP	10.510	*	10.4m	16.926	*	20.8m
	OR-Tools	9.802	1.159%	10.4m	14.933	3.853%	20.8m		OR-Tools	10.519	0.078%	10.4m	17.027	0.583%	20.8m
	MTPOMO	10.033	3.564%	2s	15.082	4.917%	8s		MTPOMO	10.667	1.472%	2s	17.421	2.896%	9s
	MVMoE	10.008	3.303%	3s	15.036	4.600%	11s		MVMoE	10.659	1.396%	3s	17.387	2.700%	12s
	RF-POMO	9.996	3.173%	2s	15.016	4.465%	8s		RF-POMO	10.657	1.376%	2s	17.392	2.725%	9s
	RF-MoE	9.980	3.014%	3s	14.973	4.165%	11s		RF-MoE	10.673	1.533%	3s	17.387	2.698%	13s
	RF-TE	9.978	2.996%	2s	14.942	3.950%	8s		RF-TE	10.652	1.328%	2s	17.326	2.341%	9s
	CaDA	9.960	2.798%	3s	14.963	4.080%	13s		CaDA	10.619	1.013%	3s	17.234	1.793%	14s
	MoSES(CaDA)	9.904	2.225%	7s	14.901	3.668%	23s		MoSES(CaDA)	10.611	0.946%	8s	17.217	1.702%	26s
	PoMtVRS(MVMoE)	9.946	2.656%	4s	14.913	3.739%	17s		PoMtVRS(MVMoE)	10.629	1.105%	4s	17.301	2.188%	20s
PoMtVRS(CaDA)	9.899	2.171%	4s	14.806	2.993%	17s	PoMtVRS(CaDA)	10.593	0.775%	4s	17.175	1.443%	20s		
VRPBL	HGS-PyVRP	10.186	*	10.4m	14.779	*	20.8m	VRPBLTW	HGS-PyVRP	18.361	*	10.4m	29.026	*	20.8m
	OR-Tools	10.331	1.390%	10.4m	15.426	4.338%	20.8m		OR-Tools	18.422	0.332%	10.4m	29.830	2.770%	20.8m
	MTPOMO	10.672	4.699%	2s	15.712	6.253%	8s		MTPOMO	18.990	2.130%	3s	30.896	3.616%	9s
	MVMoE	10.607	4.072%	3s	15.628	5.692%	11s		MVMoE	18.933	1.824%	3s	30.757	3.149%	12s
	RF-POMO	10.592	3.937%	2s	15.628	5.696%	8s		RF-POMO	18.937	1.853%	2s	30.794	3.278%	9s
	RF-MoE	10.575	3.767%	3s	15.541	5.121%	11s		RF-MoE	18.956	1.956%	3s	30.807	3.321%	12s
	RF-TE	10.578	3.798%	2s	15.528	5.038%	8s		RF-TE	18.941	1.877%	2s	30.688	2.923%	9s
	CaDA	10.548	3.508%	3s	15.530	5.029%	13s		CaDA	18.852	1.402%	3s	30.534	2.407%	14s
	MoSES(CaDA)	10.517	3.193%	7s	15.478	4.705%	24s		MoSES(CaDA)	18.858	1.425%	8s	30.510	2.329%	26s
	PoMtVRS(MVMoE)	10.542	3.420%	4s	15.493	4.783%	18s		PoMtVRS(MVMoE)	18.907	1.694%	4s	30.711	2.997%	20s
PoMtVRS(CaDA)	10.494	2.966%	4s	15.351	3.832%	18s	PoMtVRS(CaDA)	18.814	1.194%	4s	30.429	2.048%	20s		
VRPBTW	HGS-PyVRP	18.292	*	10.4m	29.467	*	20.8m	VRPLTW	HGS-PyVRP	16.356	*	10.4m	25.757	*	20.8m
	OR-Tools	18.366	0.383%	10.4m	29.945	1.597%	20.8m		OR-Tools	16.441	0.499%	10.4m	26.259	1.899%	20.8m
	MTPOMO	18.639	1.876%	2s	30.435	3.278%	9s		MTPOMO	16.823	2.818%	2s	26.891	4.364%	9s
	MVMoE	18.594	1.624%	3s	30.306	2.836%	12s		MVMoE	16.747	2.365%	3s	26.744	3.794%	12s
	RF-POMO	18.601	1.669%	2s	30.343	2.967%	9s		RF-POMO	16.750	2.383%	2s	26.784	3.951%	9s
	RF-MoE	18.617	1.760%	3s	30.339	2.947%	12s		RF-MoE	16.776	2.547%	3s	26.775	3.918%	12s
	RF-TE	18.600	1.675%	2s	30.240	2.618%	9s		RF-TE	16.763	2.460%	2s	26.691	3.587%	9s
	CaDA	18.504	1.140%	3s	30.072	2.044%	14s		CaDA	16.670	1.890%	3s	26.534	2.977%	14s
	MoSES(CaDA)	18.495	1.095%	8s	30.050	1.969%	25s		MoSES(CaDA)	16.667	1.864%	8s	26.493	2.824%	25s
	PoMtVRS(MVMoE)	18.570	1.498%	4s	30.263	2.689%	20s		PoMtVRS(MVMoE)	16.716	2.167%	4s	26.708	3.647%	20s
PoMtVRS(CaDA)	18.478	0.995%	4s	29.981	1.729%	20s	PoMtVRS(CaDA)	16.634	1.664%	4s	26.433	2.581%	20s		
OVRPB	HGS-PyVRP	6.898	*	10.4m	10.335	*	20.8m	OVRPBL	HGS-PyVRP	6.899	*	10.4m	10.335	*	20.8m
	OR-Tools	6.928	0.412%	10.4m	10.577	2.315%	20.8m		OR-Tools	6.927	0.386%	10.4m	10.582	2.363%	20.8m
	MTPOMO	7.108	3.004%	2s	10.878	5.224%	8s		MTPOMO	7.112	3.056%	2s	10.883	5.272%	8s
	MVMoE	7.107	2.991%	3s	10.886	5.310%	11s		MVMoE	7.120	3.166%	3s	10.904	5.472%	11s
	RF-POMO	7.086	2.689%	2s	10.836	4.823%	8s		RF-POMO	7.087	2.695%	2s	10.837	4.835%	8s
	RF-MoE	7.080	2.613%	3s	10.806	4.526%	11s		RF-MoE	7.083	2.635%	3s	10.807	4.540%	11s
	RF-TE	7.071	2.477%	2s	10.772	4.212%	8s		RF-TE	7.075	2.515%	2s	10.779	4.268%	8s
	CaDA	7.047	2.129%	3s	10.761	4.083%	13s		CaDA	7.049	2.146%	3s	10.762	4.093%	13s
	MoSES(CaDA)	7.034	1.942%	7s	10.726	3.765%	24s		MoSES(CaDA)	7.036	1.964%	7s	10.724	3.743%	24s
	PoMtVRS(MVMoE)	7.055	2.243%	4s	10.734	3.837%	17s		PoMtVRS(MVMoE)	7.060	2.309%	4s	10.741	3.898%	18s
PoMtVRS(CaDA)	7.021	1.755%	4s	10.657	3.093%	17s	PoMtVRS(CaDA)	7.022	1.755%	4s	10.659	3.110%	18s		
OVRPBLTW	HGS-PyVRP	11.668	*	10.4m	19.156	*	20.8m	OVRPBTW	HGS-PyVRP	11.669	*	10.4m	19.156	*	20.8m
	OR-Tools	11.681	0.106%	10.4m	19.305	0.767%	20.8m		OR-Tools	11.682	0.109%	10.4m	19.303	0.757%	20.8m
	MTPOMO	11.817	1.259%	3s	19.637	2.494%	9s		MTPOMO	11.814	1.231%	3s	19.635	2.484%	9s
	MVMoE	11.808	1.181%	4s	19.607	2.334%	12s		MVMoE	11.806	1.163%	4s	19.600	2.301%	13s
	RF-POMO	11.805	1.155%	3s	19.608	2.344%	10s		RF-POMO	11.804	1.148%	3s	19.608	2.343%	10s
	RF-MoE	11.823	1.307%	4s	19.607	2.334%	13s		RF-MoE	11.823	1.300%	4s	19.606	2.327%	13s
	RF-TE	11.804	1.147%	2s	19.551	2.045%	9s		RF-TE	11.805	1.151%	2s	19.551	2.046%	9s
	CaDA	11.761	0.778%	3s	19.441	1.468%	14s		CaDA	11.759	0.757%	3s	19.441	1.472%	14s
	MoSES(CaDA)	11.761	0.781%	8s	19.440	1.470%	26s		MoSES(CaDA)	11.760	0.773%	8s	19.441	1.475%	26s
	PoMtVRS(MVMoE)	11.775	0.894%	5s	19.525	1.904%	20s		PoMtVRS(MVMoE)	11.772	0.868%	5s	19.520	1.875%	20s
PoMtVRS(CaDA)	11.744	0.637%	5s	19.395	1.224%	20s	PoMtVRS(CaDA)	11.744	0.634%	5s	19.394	1.222%	20s		
OVRPL	HGS-PyVRP	6.507	*	10.4m	9.724	*	20.8m	OVRPLTW	HGS-PyVRP	10.510	*	10.4m	16.926	*	20.8m
	OR-Tools	6.552	0.668%	10.4m	10.001	2.791%	20.8m		OR-Tools	10.497	0.114%	10.4m	17.023	0.728%	20.8m
	MTPOMO	6.719	3.229%	2s	10.214	5.000%	8s		MTPOMO	10.670	1.503%	2s	17.420	2.892%	9s
	MVMoE	6.717	3.186%	3s	10.217	5.028%	11s		MVMoE	10.659	1.395%	3s	17.384	2.684%	12s
	RF-POMO	6.701	2.951%	2s	10.180	4.662%	8s		RF-POMO	10.657	1.372%	3s	17.392	2.727%	9s
	RF-MoE	6.696	2.870%	3s	10.141	4.253%	11s		RF-MoE	10.673	1.532%	3s	17.385	2.690%	12s
	RF-TE	6.685	2.713%	2s	10.121										

Table 2. Ablation and comparison studies on the proposed PoMtVRS across 16 VRPs.

Avg.Gap	CaDA	w.o.block	w.o.Gate	w.o.Map	w.o.PO	w.o.RL-ReLD	w.o.PO-ReLD	PoMtVRS(CaDA)
VRP50	1.721%	1.579%	1.427%	1.423%	1.621%	1.714%	1.438%	1.405%
VRP100	2.844%	2.529%	2.241%	2.323%	2.453%	2.694%	2.529%	2.205%

VRPs, and (ii) CaDA, a recent state-of-the-art approach built on a unified VRP environment. The resulting variants are denoted as PoMtVRS(MVMoE) and PoMtVRS(CaDA).

Training. Following prior protocols (Berto et al., 2025; Li et al., 2025), we train on $N \in \{50, 100\}$ for 300 epochs with 100,000 online-generated instances per epoch. We use the Adam optimizer (Kingma & Ba, 2015) with a learning rate of 3×10^{-4} , a weight decay of 1×10^{-6} , a batch size of 256, and decay the learning rate by 0.1 at epochs 270 and 295. The temperature parameter α is 0.03. We conduct experiments to examine the impact of temperature parameters on PoMtVRS’s performance (Appendix C.2). MVMoE and CaDA are trained from scratch using their original architectures (Appendix B) and hyperparameters (Appendix C.3). All experiments run on an NVIDIA Tesla V100-32GB GPU. Further detailed experimental settings are provided in Appendix C.1.

Evaluation. We evaluate using the same test dataset as in (Li et al., 2025). For each VRP variant and each problem size, we use a standardized test set of 1K randomly generated instances. For all neural solvers, we apply greedy rollout strategy with $\times 8$ augmentation (Kool et al., 2019), and we report the best solution among the eight rollouts. We report the average objective, the optimality gap, and inference time. The gap is computed with respect to the best heuristic solver results (marked by "*" in Table 1).

5.1. Experimental Results

We conduct a comprehensive benchmark over all 16 VRP variants against state-of-the-art neural solvers, with the complete results summarized in Table 1. For each task block, the last two rows report PoMtVRS instantiated with MVMoE and CaDA, respectively. Across different variants and problem scales, both PoMtVRS instantiations consistently strengthen their corresponding backbones, while PoMtVRS(CaDA) achieves the best overall performance with smaller optimality gaps on most tasks.

Quantitatively, PoMtVRS(MVMoE) improves MVMoE on all 16 variants at VRP50/VRP100, reducing the average gap from 2.227%/3.680% to 1.809%/2.951% on VRP50/VRP100, respectively. More importantly, PoMtVRS(CaDA) yields larger improvements over a stronger backbone. Compared with CaDA, it reduces the average gap from 1.721% to 1.405% on VRP50 and from 2.844% to 2.205% on VRP100, with improvements ob-

served across all 16 tasks. In addition, PoMtVRS(CaDA) also outperforms the strong prior method MoSES(CaDA) in both solution quality and inference efficiency. Specifically, the average gap is further reduced from 1.515% to 1.405% on VRP50 and from 2.631% to 2.205% on VRP100, while maintaining lower inference time. This comparison shows that PoMtVRS does not rely on expensive search or heavy model augmentation to obtain better solutions. Instead, it introduces a lightweight architectural enhancement that improves the backbone solver with limited computational overhead. Overall, PoMtVRS(CaDA) achieves leading performance with a favorable balance between solution quality and efficiency, demonstrating the effectiveness of the proposed framework for unified multi-task VRP solving.

5.2. Ablation Study

Table 2 reports the ablation results of PoMtVRS and the comparison with the ReLD (Huang et al., 2025) architecture. We evaluate 6 variants: **w.o. block**, which removes the PGB block while keeping PO; **w.o. Gate**, which removes the gating mechanism in PGB while keeping PO; **w.o. Map**, which removes the state attribute mapping branch in PGB while keeping PO; **w.o. PO**, which keeps PGB but replaces PO with RL; and two ReLD-style controls, **w.o. RL-ReLD** and **w.o. PO-ReLD**, which remove PGB and adopt the ReLD configuration under the RL or PO setup, respectively.

Across both problem scales, removing PGB (w.o. block) leads to consistent performance degradation, confirming that the proposed block is important for enhancing decoder representations. Removing either the gating mechanism (w.o. Gate) or the state attribute mapping branch (w.o. Map) weakens performance, indicating that effective multi-task adaptation relies on both preference-conditioned decoder modulation and state-aware signals for gated representation refinement. Keeping PGB but replacing PO with RL (w.o. PO) also degrades performance and remains noticeably below the PoMtVRS, indicating that PGB is most effective when coupled with the PO. The contrast between w.o. block and w.o. PO further shows that neither PO nor PGB alone is sufficient to achieve the best performance. Instead, their joint use brings complementary benefits.

We further compare with ReLD to examine whether the gains come from generic architectural changes. Under the same ReLD configuration, w.o. PO-ReLD outperforms w.o. RL-ReLD, confirming the benefit of PO. However, it still remains inferior to PoMtVRS, indicating that the improve-

Table 3. Performance comparison of multi-task VRP solvers on CVRPLib datasets.

Set-X	MTPOMO	MVMoE	RF-MoE	RF-POMO	RF-TE	CaDA	MoSES(CaDA)	PoMtVRS(CaDA)
Avg. Gap($N < 251$)	6.566%	5.829%	6.754%	5.905%	5.061%	4.772%	4.724%	4.561%
Avg. Gap($251 \leq N < 501$)	11.529%	10.616%	9.217%	8.399%	8.107%	8.889%	7.948%	7.865%
Avg. Gap($501 < N \leq 1001$)	30.190%	18.918%	14.994%	13.188%	12.253%	14.199%	12.814%	12.651%
Avg. Gap	15.863%	11.693%	10.253%	9.108%	8.428%	9.230%	8.441%	8.326%

Table 4. Zero-shot results on unseen constraints across 32 VRPs.

	Aug. Gap	MTPOMO	MVMoE	RF-TE	CaDA	PoMtVRS(CaDA)		Aug. Gap	MTPOMO	MVMoE	RF-TE	CaDA	PoMtVRS(CaDA)
VRP50	MD	43.953%	43.290%	38.205%	37.571%	34.807%	VRP100	MD	45.438%	45.762%	39.854%	43.652%	39.812%
	MB	8.709%	8.182%	8.718%	8.263%	7.770%		MB	12.296%	11.738%	12.168%	11.947%	10.936%
	MDMB	52.048%	53.723%	47.698%	49.084%	43.827%		MDMB	55.570%	57.370%	52.849%	55.501%	51.281%

ments mainly arise from the targeted coupling between PO and PGB rather than arbitrary structural modifications. Overall, the ablation results show that PO and PGB are both essential, and their combination improves optimization stability, representation enhancement, and exploration efficiency. Detailed results are provided in Appendix C.4.

5.3. Generalization on Real-World Instances

To further evaluate PoMtVRS on real-world instances, we conduct experiments on the CVRPLIB benchmark. Following the standard protocol adopted by prior methods (Berto et al., 2025; Li et al., 2025), we use X-set instances (Uchoa et al., 2017) with more than 100 nodes, covering problem sizes from 101 to 1001. Table 3 reports the averaged generalization performance of PoMtVRS(CaDA) against seven representative neural solvers on CVRPLIB. For scale-wise analysis consistent with prior work, we partition the X-set into three subsets according to the instance scale: $N < 251$, $251 \leq N < 501$, and $501 < N \leq 1001$.

Overall, PoMtVRS(CaDA) consistently strengthens the CaDA backbone across all instance scales and achieves the heading performance on the full benchmark, which indicates better scalability and more robust generalization. In particular, the consistent gains on larger instances suggest better scalability and more robust generalization under realistic routing distributions. Detailed per-instance results are provided in Appendix C.5.

5.4. Generalization to Unseen Constraints

To assess zero-shot generalization beyond the training constraint space, we evaluate on VRP50 and VRP100 under three unseen settings and report the results in Table 4.

Multi-Depot (MD): vehicles depart from one of three depots and return to the same depot (Berto et al., 2025); we build 16 MD variants by adding MD to each base problem in Table 1. **Mixed Backhauls (MB):** we remove the precedence between linehaul and backhaul customers and allow arbitrary service order under capacity constraints. We form 8 MB variants by replacing backhaul with MB in backhaul-

related problems. We further evaluate their composition (**MDMB**) to induce a stronger shift from unseen constraint combinations, resulting in 8 additional variants. Across MD, MB, and MDMB, PoMtVRS(CaDA) delivers the best solution quality at both scales, consistently surpassing the CaDA backbone and other neural baselines, which confirms robust generalization to constraint shifts and their compositions.

5.5. Further Enhancement with Efficient Active Search

In addition to the main comparison, we further investigate whether PoMtVRS can benefit from test-time adaptation. Specifically, we integrate efficient active search (EAS) (Hotung et al., 2022) into PoMtVRS(CaDA) by optimizing a lightweight instance-specific residual module during inference and further adopt LoRA to reduce the search overhead. As reported in Appendix D, experiments on 1K test instances across all 16 VRP variants show that EAS consistently improves solution quality, while the LoRA-based variant preserves most of the gains with substantially lower search cost. Notably, PoMtVRS(CaDA) with EAS or EAS-LoRA continues to outperform the corresponding CaDA-based counterparts under the same search setting, demonstrating that the proposed architecture remains effective and complementary to test-time search.

6. Conclusion, Limitation, and Future Work

This paper proposes PoMtVRS, a plug-and-play preference learning framework for multi-task VRPs that improves decoder adaptation under frequent constraint switches and strengthens exploration beyond scalarized rewards. Extensive experiments show that PoMtVRS integrates seamlessly with mainstream backbones and achieves leading performance and generalization, while ablation studies also confirm the effectiveness of the preference-gated block and the PO algorithm working together. Limitations remain in handling more complex real-world constraint systems and larger problem scales. Future work will extend PoMtVRS to richer constraints and broader real-world distributions, and further improve efficiency for large-scale deployment.

Acknowledgments

This work was supported in part by the National Natural Science Foundation of China under Grant 62372081, the Liaoning Provincial Central-Guided Local Sci-Tech Development Program under Grant 2025JH6/101000005, the Dalian Science and Technology Innovation Fund under Grant 2024JJ12GX020, the Liaoning Provincial Natural Science Foundation Program under Grant 2024-MSBA-05 and the 111 Project under Grant D23006. This work is also supported by National Natural Science Foundation of China under grant No. 52472461 from the Second Research Institute of Civil Aviation Administration of China.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

- Bello, I., Pham, H., Le, Q. V., Norouzi, M., and Bengio, S. Neural combinatorial optimization with reinforcement learning. In *ICLR Workshop Track*, 2017.
- Bengio, Y., Lodi, A., and Prouvost, A. Machine learning for combinatorial optimization: a methodological tour d’horizon. *European Journal of Operational Research*, 290(2):405–421, 2021.
- Berto, F., Hua, C., Park, J., Kim, M., Kim, H., Son, J., Kim, H., Kim, J., and Park, J. RL4CO: a unified reinforcement learning for combinatorial optimization library. In *NeurIPS 2023 Workshop: New Frontiers in Graph Learning*, 2023.
- Berto, F., Hua, C., Zepeda, N. G., Hottung, A., Wouda, N., Lan, L., Park, J., Tierney, K., and Park, J. RouteFinder: Towards Foundation Models for Vehicle Routing Problems. *Transactions on Machine Learning Research*, 2025. ISSN 2835-8856.
- Bi, J., Ma, Y., Wang, J., Cao, Z., Chen, J., Sun, Y., and Chee, Y. M. Learning generalizable models for vehicle routing problems via knowledge distillation. In *Neural Information Processing Systems (NeurIPS)*, 2022.
- Chalumeau, F., Surana, S., Bonnet, C., Grinsztajn, N., Pretorius, A., Laterre, A., and Barrett, T. D. Combinatorial optimization with policy adaptation using latent space search. In *Neural Information Processing Systems (NeurIPS)*, 2023.
- Chen, J., Zhang, Z., Cao, Z., Wu, Y., Ma, Y., Ye, T., and Wang, J. Neural multi-objective combinatorial optimization with diversity enhancement. In *Neural Information Processing Systems (NeurIPS)*, 2023.
- Chen, X. and Tian, Y. Learning to perform local rewriting for combinatorial optimization. In *Neural Information Processing Systems (NeurIPS)*, volume 32, 2019.
- d O Costa, P. R., Rhuggenaath, J., Zhang, Y., and Akcay, A. Learning 2-opt heuristics for the traveling salesman problem via deep reinforcement learning. In *Asian Conference on Machine Learning*, pp. 465–480. PMLR, 2020.
- Dong, H., Xiong, W., Goyal, D., Zhang, Y., Chow, W., Pan, R., Diao, S., Zhang, J., SHUM, K., and Zhang, T. RAFT: Reward ranked finetuning for generative foundation model alignment. *Transactions on Machine Learning Research*, 2023. ISSN 2835-8856.
- Drakulic, D., Michel, S., Mai, F., Sors, A., and Andreoli, J. BQ-NCO: bisimulation quotienting for efficient neural combinatorial optimization. In *Neural Information Processing Systems (NeurIPS)*, volume 36, pp. 77416–77429, 2023.
- Drakulic, D., Michel, S., and Andreoli, J.-M. GOAL: A Generalist Combinatorial Optimization Agent Learning. In *International Conference on Learning Representations (ICLR)*, 2025.
- Fan, M., Zhou, J., Zhang, Y., Wu, Y., Chen, J., and Sartoretti, G. Preference-driven multi-objective combinatorial optimization with conditional computation. In *Neural Information Processing Systems (NeurIPS)*, volume 38, pp. 11594–11627, 2026.
- Ge, Y., Li, H., and Tuzhilin, A. Route recommendations for intelligent transportation services. *IEEE Transactions on Knowledge and Data Engineering*, 33(3):1169–1182, 2019.
- Goh, Y. L., Cao, Z., Ma, Y., Zhou, J., Dupty, M. H., and Lee, W. S. Shield: Multi-task multi-distribution vehicle routing solver with sparsity and hierarchy. In *International Conference on Machine Learning (ICML)*, pp. 19679–19711, 2025.
- Gui, S., Liu, S., Wang, X., and Cao, Z. Chain-of-context learning: Dynamic constraint understanding for multi-task VRPs. In *International Conference on Learning Representations (ICLR)*, 2026.
- Helsgaun, K. An extension of the lin-kernighan-helsgaun tsp solver for constrained traveling salesman and vehicle routing problems. *Roskilde: Roskilde University*, 12: 966–980, 2017.
- Hottung, A. and Tierney, K. Neural large neighborhood search for the capacitated vehicle routing problem. In

- European Conference on Artificial Intelligence*, pp. 443–450, 2020.
- Hottung, A., Kwon, Y.-D., and Tierney, K. Efficient active search for combinatorial optimization problems. In *International Conference on Learning Representation (ICLR)*, 2022.
- Hottung, A., Mahajan, M., and Tierney, K. PolyNet: Learning diverse solution strategies for neural combinatorial optimization. *arXiv preprint arXiv:2402.14048*, 2024.
- Hou, Q., Yang, J., Su, Y., Wang, X., and Deng, Y. Generalize learned heuristics to solve large-scale vehicle routing problems in real-time. In *International Conference on Learning Representations (ICLR)*, 2023.
- Huang, Z., Zhou, J., Cao, Z., and Xu, Y. Rethinking light decoder-based solvers for vehicle routing problems. In *International Conference on Learning Representations (ICLR)*, 2025.
- Hudson, B., Li, Q., Malencia, M., and Prorok, A. Graph neural network guided local search for the traveling salesperson problem. In *International Conference on Learning Representations (ICLR)*, 2022.
- Jiang, X., Wu, Y., Wang, Y., and Zhang, Y. Bridging large language models and optimization: A unified framework for text-attributed combinatorial optimization. *arXiv preprint arXiv:2408.12214*, 2024.
- Kim, M., Park, J., and Park, J. Sym-NCO: Leveraging symmetry for neural combinatorial optimization. In *Neural Information Processing Systems (NeurIPS)*, 2022.
- Kingma, D. and Ba, J. Adam: A Method for Stochastic Optimization. In *International Conference on Learning Representations (ICLR)*, San Diego, CA, USA, 2015.
- Konstantakopoulos, G. D., Gayialis, S. P., and Kechagias, E. P. Vehicle routing problem and related algorithms for logistics distribution: A literature review and classification. *Operational research*, 22(3):2033–2062, 2022.
- Kool, W., van Hoof, H., and Welling, M. Attention, learn to solve routing problems! In *International Conference on Learning Representations (ICLR)*, 2019.
- Kwon, Y.-D., Choo, J., Kim, B., Yoon, I., Gwon, Y., and Min, S. POMO: Policy optimization with multiple optima for reinforcement learning. In *Neural Information Processing Systems (NeurIPS)*, volume 33, pp. 21188–21198, 2020.
- Kwon, Y.-D., Choo, J., Yoon, I., Park, M., Park, D., and Gwon, Y. Matrix encoding networks for neural combinatorial optimization. In *Neural Information Processing Systems (NeurIPS)*, volume 34, pp. 5138–5149, 2021.
- Laporte, G. and Nobert, Y. A branch and bound algorithm for the capacitated vehicle routing problem. *Operations-Research-Spektrum*, 5(2):77–85, 1983.
- Li, H., Liu, F., Zheng, Z., Zhang, Y., and Wang, Z. Cada: Cross-problem routing solver with constraint-aware dual-attention. In *International Conference on Machine Learning (ICML)*, 2025.
- Li, J., Ma, Y., Gao, R., Cao, Z., Lim, A., Song, W., and Zhang, J. Deep reinforcement learning for solving the heterogeneous capacitated vehicle routing problem. *IEEE Transactions on Cybernetics*, 52(12):13572–13585, 2021.
- Liao, Z., Chen, J., Wang, D., Zhang, Z., and Wang, J. BOPO: Neural combinatorial optimization via best-anchored and objective-guided preference optimization. In *International Conference on Machine Learning (ICML)*, 2025.
- Lin, Z., Wu, Y., Zhou, B., Cao, Z., Song, W., Zhang, Y., and Jayavelu, S. Cross-problem learning for solving vehicle routing problems. In *International Joint Conferences on Artificial Intelligence Organization (IJCAI)*, pp. 6958–6966, 2024.
- Liu, F., Lin, X., Wang, Z., Zhang, Q., Xialiang, T., and Yuan, M. Multi-task learning for routing problem with cross-problem zero-shot generalization. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pp. 1898–1908, 2024.
- Liu, S., Cao, Z., Feng, S., and Ong, Y.-S. A mixed-curvature based pre-training paradigm for multi-task vehicle routing solver. In *International Conference on Machine Learning (ICML)*, 2025.
- Lu, H., Zhang, X., and Yang, S. A learning-based iterative method for solving vehicle routing problems. In *International conference on learning representations (ICLR)*, 2020.
- Luo, F., Lin, X., Liu, F., Zhang, Q., and Wang, Z. Neural combinatorial optimization with heavy decoder: Toward large scale generalization. In *Neural Information Processing Systems (NeurIPS)*, volume 36, pp. 8845–8864, 2023.
- Luo, F., Lin, X., Wu, Y., Wang, Z., Xialiang, T., Yuan, M., and Zhang, Q. Boosting neural combinatorial optimization for large-scale vehicle routing problems. In *International Conference on Learning Representations (ICLR)*, 2025.
- Ma, Y., Cao, Z., and Chee, Y. M. Learning to search feasible and infeasible regions of routing problems with flexible neural k-opt. In *Neural Information Processing Systems (NeurIPS)*, 2023.

- Meng, D., Cao, Z., Gao, J., Wu, Y., and Hou, Y. Uniteformer: Unifying node and edge modalities in transformers for vehicle routing problems. In *Neural Information Processing Systems (NeurIPS)*, 2025a.
- Meng, D., Cao, Z., Wu, Y., Hou, Y., Ge, H., and Zhang, Q. Eformer: An effective edge-based transformer for vehicle routing problems. In *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence, IJCAI-25*, pp. 8582–8590. International Joint Conferences on Artificial Intelligence Organization (IJCAI), 2025b.
- Meng, Y., Xia, M., and Chen, D. Simpo: Simple preference optimization with a reference-free reward. *Neural Information Processing Systems (NeurIPS)*, 37:124198–124235, 2024.
- Nazari, M., Oroojlooy, A., Snyder, L., and Takác, M. Reinforcement learning for solving the vehicle routing problem. In *Neural Information Processing Systems (NeurIPS)*, volume 31, 2018.
- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., et al. Training language models to follow instructions with human feedback. In *Neural Information Processing Systems (NeurIPS)*, volume 35, pp. 27730–27744, 2022.
- Pan, M., Lin, G., Luo, Y.-W., Zhu, B., Dai, Z., Sun, L., and Yuan, C. Preference optimization for combinatorial optimization problems. In *International Conference on Machine Learning (ICML)*, 2025a.
- Pan, Y., Cao, Z., Gu, C., Liu, L., Zhao, P., Chen, Y., and Lin, F. Multi-task vehicle routing solver via mixture of specialized experts under state-decomposable mdp. In *Neural Information Processing Systems (NeurIPS)*, 2025b.
- Perron, L. and Furnon, V. OR-Tools. Google, 2023.
- Qiu, Z., Wang, Z., Zheng, B., Huang, Z., Wen, K., Yang, S., Men, R., Yu, L., Huang, F., Huang, S., et al. Gated attention for large language models: Non-linearity, sparsity, and attention-sink-free. In *Neural Information Processing Systems (NeurIPS)*, volume 38, pp. 100092–100118, 2026.
- Rafailov, R., Sharma, A., Mitchell, E., Manning, C. D., Ermon, S., and Finn, C. Direct preference optimization: Your language model is secretly a reward model. In *Neural Information Processing Systems (NeurIPS)*, volume 36, pp. 53728–53741, 2023.
- Song, F., Yu, B., Li, M., Yu, H., Huang, F., Li, Y., and Wang, H. Preference ranking optimization for human alignment. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pp. 18990–18998, 2024.
- Stiennon, N., Ouyang, L., Wu, J., Ziegler, D., Lowe, R., Voss, C., Radford, A., Amodei, D., and Christiano, P. F. Learning to summarize with human feedback. In *Neural Information Processing Systems (NeurIPS)*, volume 33, pp. 3008–3021, 2020.
- Uchoa, E., Pecin, D., Pessoa, A., Poggi, M., Vidal, T., and Subramanian, A. New benchmark instances for the capacitated vehicle routing problem. *European Journal of Operational Research*, 257(3):845–858, 2017.
- Vidal, T. Hybrid genetic search for the cvrp: Open-source implementation and swap* neighborhood. *Computers & Operations Research*, 140:105643, 2022. ISSN 0305-0548. doi: <https://doi.org/10.1016/j.cor.2021.105643>.
- Vidal, T., Crainic, T. G., Gendreau, M., Lahrichi, N., and Rei, W. A hybrid genetic algorithm for multidepot and periodic vehicle routing problems. *Operations Research*, 60(3):611–624, 2012.
- Vinyals, O., Fortunato, M., and Jaitly, N. Pointer networks. In *Neural Information Processing Systems (NeurIPS)*, volume 28, 2015.
- Wang, C. and Yu, T. Efficient training of multi-task neural solver with multi-armed bandits. *Transactions on Machine Learning Research*, 2025.
- Williams, R. J. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning*, 8(3):229–256, 1992.
- Wouda, N. A., Lan, L., and Kool, W. PyVRP: a high-performance VRP solver package. *INFORMS Journal on Computing*, 36(4):943–955, 2024. doi: 10.1287/ijoc.2023.0055.
- Wu, Y., Song, W., Cao, Z., Zhang, J., and Lim, A. Learning improvement heuristics for solving routing problems. *IEEE transactions on neural networks and learning systems*, 33(9):5057–5069, 2021.
- Wu, Y., Yu, Y., Wu, L., Feng, T., Zhang, L., Wang, Z., and Gao, J. Deep reinforcement learning approach to solving clustered vehicle routing problems. *Transportation Research Part E: Logistics and Transportation Review*, 209:104742, 2026.
- Xiao, W., Wang, Z., Gan, L., Zhao, S., He, W., Tuan, L. A., Chen, L., Jiang, H., Zhao, Z., and Wu, F. A comprehensive survey of datasets, theories, variants, and applications in direct preference optimization. *arXiv e-prints*, pp. arXiv–2410, 2024.
- Xin, L., Song, W., Cao, Z., and Zhang, J. NeuroLKH: Combining deep learning model with lin-kernighan-helsgaun heuristic for solving the traveling salesman problem. In

Neural Information Processing Systems (NeurIPS), volume 34, pp. 7472–7483, 2021.

Xu, J., Lee, A., Sukhbaatar, S., and Weston, J. Some things are more cringe than others: Iterative preference optimization with the pairwise cringe loss. *arXiv preprint arXiv:2312.16682*, 2023.

Zhang, C., Wu, Y., Ma, Y., Song, W., Le, Z., Cao, Z., and Zhang, J. A review on learning to solve combinatorial optimisation problems in manufacturing. *IET Collaborative Intelligent Manufacturing*, 5(1):e12072, 2023.

Zheng, Z., Zhou, C., Tong, X., Yuan, M., and Wang, Z. Udc: A unified neural divide-and-conquer framework for large-scale combinatorial optimization problems. In *Neural Information Processing Systems (NeurIPS)*, volume 37, pp. 6081–6125, 2024.

Zhou, J., Wu, Y., Cao, Z., Song, W., Zhang, J., and Chen, Z. Learning large neighborhood search for vehicle routing in airport ground handling. *IEEE Transactions on Knowledge and Data Engineering*, 2023a.

Zhou, J., Wu, Y., Song, W., Cao, Z., and Zhang, J. Towards omni-generalizable neural methods for vehicle routing problems. In *International conference on machine learning (ICML)*, pp. 42769–42789. PMLR, 2023b.

Zhou, J., Cao, Z., Wu, Y., Song, W., Ma, Y., Zhang, J., and Xu, C. Mvmoe: multi-task vehicle routing solver with mixture-of-experts. In *International Conference on Machine Learning (ICML)*, pp. 61804–61824, 2024.

A. Multi-Task VRPs Details

A.1. VRP Variants

A VRP instance of size N on a complete graph $\mathcal{G} = \{V, E\}$, where $V = \{v_0, v_1, \dots, v_N\} (|V| = N + 1)$ consists of a depot v_0 and N customers $\{v_1, \dots, v_N\}$. Each node $v_i \in V$ has 2D coordinates (x_i, y_i) . Each customer additionally carries constraint attributes c_i (e.g., demand/backhaul amount, time-window parameters). The edge set is $E = \{e(v_i, v_j) \mid 0 \leq i \neq j \leq N\}$, and each edge induces a travel cost e_{ij} (distance or travel time). Vehicles are homogeneous with capacity Q . A solution τ is a set of sub-routes. Each sub-route departs from the depot, visits a subset of customers, and must satisfy feasibility: every customer is served exactly once, and all route-level constraints are respected throughout execution (e.g., cumulative demand never exceeds Q). We define the objective $c(\tau)$ as the total Euclidean length and seek $\tau^* = \arg \min_{\tau} c(\tau)$, i.e., the minimum-cost feasible routing plan.

We take CVRP as the base variant, characterized by the capacity constraint (C). Other fundamental VRP variants are obtained by adding extra constraints, e.g., OVRP by enabling open routes (O), VRPB by imposing route Backhauls (B), VRPL by imposing route duration limits (L), and VRPTW by imposing route Time Window (TW). We use five basic constraints to construct the task family. Combining any subset of the four additional constraints (O)/(B)/(L)/(TW) on top of (C) yields 16 VRP variants (2^4 possible combinations): CVRP, OVRP, VRPB, VRPL, VRPTW, OVRPTW, OVRPB, OVRPL, VRPBL, VRPBTW, VRPLTW, OVRPBL, OVRPBTW, OVRPLTW, VRPBLTW, OVRPBLTW.

Capacity constraint (C) [$q \in [0, Q]$]. Each customer v_i has a delivery demand q_i served by vehicles with fixed capacity $Q > 0$. The vehicle load must never exceed Q .

Open Routes (O) [$o \in \{0, 1\}$]. A vehicle is not required to return to the depot after finishing its assigned customers. A sub-route may terminate at the last visited customer.

Backhauls (B) [$b \in [0, Q]$]. We generalize demands via b_i , allowing both deliveries and pickups. Linehaul customers satisfy $b_i > 0$ (deliver from depot), while backhaul customers satisfy $b_i < 0$ (pickup to depot). A route may serve both types, but all linehaul customers must precede any backhaul customer to avoid reloading and capacity conflicts.

Duration Limit (L) [$l \in [0, L]$]. Each sub-route is constrained by an upper bound on total travel duration (or length), balancing workload across vehicles.

Time Window (TW) [$e, s, f \in [0, T]^3$]. Each customer v_i is associated with a service time window $[e_i, f_i]$ and a service duration s_i . The vehicle must arrive no later than f_i . If it arrives early, it must wait until e_i to start service, ensuring the service start time lies within $[e_i, f_i]$.

A.2. VRP Instance Generation

This section specifies the instance generation pipeline of the unified VRP environment (Berto et al., 2025), covering node attributes and global constraint attributes. The environment naturally supports multi-depot routing, where m denotes the

Table 5. 16 VRP variants with five constraints.

	Capacity (C)	Open Route (O)	Backhaul (B)	Duration Limit (L)	Time Window (TW)
CVRP	✓				
OVRP	✓	✓			
VRPB	✓		✓		
VRPL	✓			✓	
VRPTW	✓				✓
OVRPTW	✓	✓			✓
OVRPB	✓	✓	✓		
OVRPL	✓	✓		✓	
VRPBL	✓		✓	✓	
VRPBTW	✓		✓		✓
VRPLTW	✓			✓	✓
OVRPBL	✓	✓	✓	✓	
OVRPBTW	✓	✓	✓		✓
OVRPLTW	✓	✓		✓	✓
VRPBLTW	✓		✓	✓	✓
OVRPBLTW	✓	✓	✓	✓	✓

number of depots and n the number of customers. For fair comparison with mainstream neural baselines, the main paper focuses on the single-depot setting ($m = 1$).

Locations. For each instance, we independently sample $m + n$ 2D coordinates from the unit square:

$$(x_i, y_i) \sim U(0, 1), \quad i \in \{0, \dots, m + n - 1\}, \quad (22)$$

where $\{(x_i, y_i)\}_{i=0}^{m-1}$ denote the depot coordinates, $\{(x_i, y_i)\}_{i=m}^{m+n-1}$ represent the coordinates of the customers. Distances are computed by Euclidean metrics, $d_{ij} = \|(x_i, y_i) - (x_j, y_j)\|_2$. The vehicle speed is set to 1.0, so travel time equals distance.

Vehicle capacity (C). We consider homogeneous vehicles and use a single capacity scalar C shared by all routes within an instance. Following common practice in multi-task VRP benchmarks, C is set as a deterministic function of n :

$$C = \begin{cases} 30 + \left\lfloor \frac{n}{5} \right\rfloor + \left\lfloor \frac{1000}{5} + \frac{n - 1000}{33.3} \right\rfloor, & n > 1000, \\ 30 + \left\lfloor \frac{n}{5} \right\rfloor, & 20 < n \leq 1000, \\ 30, & \text{otherwise.} \end{cases} \quad (23)$$

When needed for stable learning, demands are normalized by C and the normalized capacity is set to 1.

Open Route (O). The open-route attribute is indicated by a binary variable $o \in \{0, 1\}$. When $o=1$, the route can terminate at the last served customer (open route), when $o=0$, the vehicle must return to a depot (closed route). The effective proportion of open route instances during training/evaluation is controlled by the sampling recipe, enabling consistent multi-task mixtures without changing the underlying instance format.

Distance Limit (L). The distance-limit constraint assigns a per-route upper bound ρ such that each sub-route starting from a depot must have length at most ρ . To guarantee feasibility, ρ is sampled from an interval determined by the farthest customers:

$$\rho \sim U(2d_{\max}, \rho_{\max}), \quad d_{\max} = \begin{cases} \max_{i \in \mathcal{C}} d_{0i}, & m = 1, \\ \min_{j \in \mathcal{D}} \max_{i \in \mathcal{C}} d_{ji}, & m > 1, \end{cases} \quad (24)$$

where $\mathcal{D}$ and $\mathcal{C}$ denote depots and customers, respectively, and $\rho_{\max}$ is a predefined upper bound (e.g., 3.0). This strategy ensures round-trip reachability from at least one depot while allowing varying constraint tightness. If the distance limit is inactive, we set $\rho = +\infty$.

Linehaul and Backhaul Demands (B). For the depot nodes, both linehaul and backhaul demands are set to zero. For each customer v_i ($i \geq m$), we first sample two nonnegative integers $\tilde{q}_i^{\text{LH}}, \tilde{q}_i^{\text{BH}} \sim \text{Unif}\{1, 2, \dots, 9\}$, and then draw a Bernoulli indicator $z_i \sim \text{Bernoulli}(p_{\text{BH}})$ with $p_{\text{BH}}=0.2$ to assign the customer type:

$$(q_i^{\text{LH}}, q_i^{\text{BH}}) = \begin{cases} (\tilde{q}_i^{\text{LH}}, 0), & z_i = 0 \quad (\text{linehaul}), \\ (0, \tilde{q}_i^{\text{BH}}), & z_i = 1 \quad (\text{backhaul}). \end{cases} \quad (25)$$

Backhaul demands are activated only for variants that include the backhaul constraint. Otherwise, we set $q_i^{\text{BH}}=0$ for all customers and use q_i^{LH} as the standard demand.

Time Windows (TW). Time windows assign each customer v_i ($i \geq m$) an interval $[e_i, l_i]$ and a service duration s_i . We set the vehicle speed to 1.0 so that travel times numerically match Euclidean distances. For the depot, we use $e_0 = 0$, $s_0 = 0$, and an overall horizon $l_0 = T$. We fix the depot time window to $[e_0, l_0] = [0, T]$ with $T = 4.6$ and set the depot service time to $s_0 = 0$. For each customer $i \in \mathcal{C}$, we sample

$$s_i \sim U(0.15, 0.18), \quad \Delta t_i \sim U(0.18, 0.20), \quad (26)$$

where Δt_i is the window length. Let $d_i = d_{0i}$ for $m=1$, and $d_i = \min_{j \in \mathcal{D}} d_{ji}$ for $m > 1$. To ensure that the simple tour $(0, i, 0)$ remains feasible, we compute an upper bound for the window start time:

$$u_i^{\text{up}} = \frac{T - s_i - \Delta t_i}{d_i} - 1, \quad (27)$$

then draw $u_i \sim U(0, 1)$ and set the start times of the time windows e_i :

$$e_i = (1 + (u_i^{\text{up}} - 1)u_i) d_i, \quad l_i = e_i + \Delta t_i. \quad (28)$$

If the time-window constraint is inactive, we set $e_i = 0$, $l_i = +\infty$, and $s_i = 0$ for all customers. During decoding, feasibility is enforced via action masking (e.g., requiring predicted arrival times not exceeding l_i), and return-to-depot feasibility is additionally checked under the closed-route setting.

Variant Composition. The above attributes are composed modularly. By activating different subsets of constraint flags, the environment instantiates multiple VRP variants within a single unified multi-task setting for both training and evaluation.

A.3. Feasible Action Space

To derive the feasible action space in the unified modular VRP environment, we construct a boolean action mask $m_t(i) \in \{0, 1\}$ at each decoding step t , where $m_t(i) = 1$ indicates that selecting node v_i next is feasible. Let the vehicle be at node v_{τ_t} , with current time w_t^{cur} and cumulative length of the current sub-route l_t^{cur} . The Euclidean distance is denoted by $\text{Dist}(\cdot, \cdot)$. The vehicle speed is fixed to 1.0, and time windows are normalized accordingly, so the travel time is numerically identical to the Euclidean distance. A candidate action i is feasible only if it passes the following checks.

(1) Visit-once and depot-action constraints. Each customer can be visited at most once, hence visited customers are always masked afterwards. To avoid degenerate self-loops, if the previous action is a depot, the next action cannot be a depot. Moreover, when the vehicle is currently at a depot while some customers remain unserved, depot actions are disallowed to prevent the decoder from stalling. Once all customers are served, depot selection is enabled to terminate decoding.

(2) Time-window feasibility. When the time-window constraint is active, each customer v_i has a service window $[w_i^{\text{beg}}, w_i^{\text{end}}]$ and a service duration w_i^{dur} . The arrival time at v_i is $a_t(i) = w_t^{\text{cur}} + \text{Dist}(v_{\tau_t}, v_i)$, and the service start time is $s_t(i) = \max(a_t(i), w_i^{\text{beg}})$. Action i is feasible only if

$$s_t(i) + w_i^{\text{dur}} \leq w_i^{\text{end}}, \quad (29)$$

otherwise it is masked out.

(3) Distance-limit feasibility. When the distance-limit constraint is active, the current sub-route length cannot exceed l^{dur} . Thus, action i is feasible only if

$$l_t^{\text{cur}} + \text{Dist}(v_{\tau_t}, v_i) \leq l^{\text{dur}}. \quad (30)$$

(4) Return-to-depot feasibility for closed routes. When the open-route constraint is inactive (i.e., routes must return to the depot), we additionally require that after visiting v_i the vehicle can finish service and return to the starting depot of the current sub-route before the system end time w_0^{end} . Denote this starting depot by $v_{d(t)}$ (for the single-depot case, $v_{d(t)} = v_0$). Then action i must satisfy the return-time feasibility

$$\max(w_t^{\text{cur}} + \text{Dist}(v_{\tau_t}, v_i), w_i^{\text{beg}}) + w_i^{\text{dur}} + \text{Dist}(v_i, v_{d(t)}) \leq w_0^{\text{end}}. \quad (31)$$

If the distance-limit constraint is also active, we further enforce the return-distance feasibility

$$l_t^{\text{cur}} + \text{Dist}(v_{\tau_t}, v_i) + \text{Dist}(v_i, v_{d(t)}) \leq l^{\text{dur}}. \quad (32)$$

When the open-route constraint is active, the return feasibility checks above are removed, while the local TW and distance-limit checks remain enforced.

(5) Capacity and linehaul-backhaul precedence. When capacity is active, selecting v_i requires that its demand does not exceed the remaining capacity. For variants with both linehaul and backhaul, we maintain separate remaining capacities Q_t^{LH} and Q_t^{BH} , and check $q_i^{\text{LH}} \leq Q_t^{\text{LH}}$ or $q_i^{\text{BH}} \leq Q_t^{\text{BH}}$ depending on the customer type. In addition, we impose within-sub-route precedence: if there exist unserved linehaul customers in the current sub-route, all backhaul customers are masked. Once the sub-route enters the backhaul phase, remaining linehaul customers are disallowed within that sub-route, preventing LB precedence violations.

(6) Multi-depot extension. For the multi-depot setting ($m > 1$), the rules remain unchanged except that depot-related terms in the return feasibility are evaluated with respect to the starting depot $v_{d(t)}$ of the current sub-route. We also mask evidently invalid depot actions (e.g., depots that cannot serve any remaining customers) to ensure a non-stalling decoding process.

B. Backbone Method Details

B.1. MVMoE Backbone: Multi-Task VRP Solver with Mixture-of-Experts

MVMoE builds upon a classic Transformer construction policy as a shared backbone, enabling cross-task reuse through a unified feature space. It further increases model capacity by injecting Mixture-of-Experts (MoE) layers, while controlling the computational overhead via sparse and hierarchical gating.

Mixture-of-Experts (MoE). To enhance multi-task modeling, MVMoE augments the shared backbone with MoE layers. A typical MoE layer consists of a routing network and m experts $\{E_j(\cdot)\}_{j=1}^m$. Given a single input x , the router produces a sparse gating vector $g(x) \in \mathbb{R}^m$ (e.g., via Top- K selection), and the MoE output is computed as:

$$\text{MoE}(x) = \sum_{j=1}^m g_j(x) E_j(x), \quad (33)$$

where only a small subset of experts is activated per token. In MVMoE, MoE is used to replace the dense FFN in the encoder to encourage expert-wise specialization for graph representation learning. In addition, MVMoE parameterizes key decoder mappings (e.g., the output projection after MHA) with MoE to improve policy expressiveness under heterogeneous constraints. The router can be made task-aware by conditioning on a task embedding, enabling task-specific expert selection within a single shared model.

Encoder with MoE. MVMoE adopts a Transformer encoder to embed node features and model global interactions. Let $X_l \in \mathbb{R}^{n \times d}$ denote the token matrix at the l -th layer, where n is the number of nodes and d is the embedding dimension. Each layer follows a residual pre-norm formulation:

$$\tilde{X}_l = \text{IN}(X_l + \text{MHA}(X_l)), \quad (34)$$

$$X_{l+1} = \text{IN}(\tilde{X}_l + \text{MoE}(\tilde{X}_l)), \quad (35)$$

where $\text{IN}(\cdot)$ denotes instance normalization and $\text{MHA}(\cdot)$ is multi-head attention. Compared with a standard dense Transformer, the FFN in each encoder layer is replaced by an MoE module, thereby increasing capacity with limited additional computation due to sparse routing.

Decoder with MoE. The decoder constructs a solution in an autoregressive manner. At step t , it forms a context embedding c_t by aggregating the depot embedding, the embedding of the last selected node, and the current resource state. Based on c_t , the decoder queries the encoded node embeddings $H = \{h_i\}_{i=1}^n$ and computes scores over candidate nodes. With an infeasibility mask applied, the next-node distribution is given by:

$$u_{t,i} = \text{Score}(c_t, h_i), \quad (36)$$

$$p_t = \text{softmax}(\text{Mask}(u_t)), \quad (37)$$

where $\text{Mask}(\cdot)$ sets infeasible logits to $-\infty$ before softmax. To further expand capacity, MVMoE replaces the dense output projection following multi-head attention with an MoE module. Concretely, letting $\{\text{head}_r\}_{r=1}^M$ be the attention heads, the projected query is computed as

$$q_t = \text{MoE}(\text{Concat}(\text{head}_1, \dots, \text{head}_M)), \quad (38)$$

which provides conditional specialization in the decoder while maintaining tractable complexity through sparse routing.

Gating mechanisms: base sparse gating and hierarchical gating. For sparse routing, MVMoE adopts *input-choice* gating: each token is assigned to its top- k experts according to gate scores. Given token matrix $X \in \mathbb{R}^{n \times d}$, gate logits are computed as $A = XW_G$, and each token selects the top- k entries in its corresponding row of A , followed by a softmax over the selected experts to obtain sparse gates. To better trade off performance and efficiency in decoding, MVMoE further introduces *hierarchical gating*. A problem-level gate G_1 first decides whether to use a sparse MoE branch or a dense branch:

$$\bar{x} = \text{Pool}(X), \quad (39)$$

$$r = \text{softmax}(\bar{x}W_{G_1}), \quad (40)$$

where $\text{Pool}(\cdot)$ aggregates node tokens into a problem representation. If G_1 selects the sparse branch, a node-level gate G_2 routes tokens to experts as above. Otherwise, a dense transformation $D(X)$ is applied. This design reduces the average number of routed tokens and expert activations while preserving robustness across diverse and out-of-distribution instances.

Training objective. MVMoE is trained with RL to minimize the expected cost. The overall loss combines (i) a REINFORCE-style policy-gradient objective $\mathcal{L}_a$ and (ii) a load-balancing regularizer $\mathcal{L}_b$ to prevent expert collapse:

$$\mathcal{L} = \mathcal{L}_a + \alpha \mathcal{L}_b, \quad (41)$$

where α controls the strength of load balancing. This objective jointly promotes strong routing performance and effective utilization of multiple experts under sparse or hierarchical gating.

B.2. CaDA Backbone: Cross-Problem Routing Solver with Constraint-Aware Dual-Attention

CaDA (Li et al., 2025) targets multi-task VRPs with an end-to-end encoder-decoder policy. The encoder maps an VRP instance to node representations, and the decoder constructs a feasible solution autoregressively by selecting the next node under constraint-induced feasibility masks. To better accommodate heterogeneous constraint combinations, CaDA injects a constraint-aware prompt into the encoder and employs a dual-attention encoder to capture both global interactions and salient sparse dependencies.

Constraint prompt. To explicitly encode the activated constraints, CaDA represents the constraint configuration by a multi-hot vector $V \in \mathbb{R}^5$ and transforms it via a multi-layer perceptron (MLP) into a prompt embedding $P^{(0)}$:

$$P^{(0)} = \text{LayerNorm}(VW_a + b_a)W_b + b_b, \quad (42)$$

where W_a, b_a, W_b and b_b are learnable parameters. The prompt is concatenated with node embeddings and propagated through the encoder, thereby injecting constraint information into representation learning.

Dual-attention encoder. CaDA stacks L encoder layers with two parallel branches: a *global* branch and a *sparse* branch. The global branch uses standard MHA and a feed-forward layer (SwiGLU), together with RMSNorm and residual connections, to preserve full-graph information flow. The sparse branch follows a similar attention and FFN structure but replaces dense attention with Top- k sparse attention. For each query, only the Top- k attention scores are retained while the rest are masked before softmax, emphasizing highly related node pairs and suppressing redundant interactions. At the end of each layer, the two branches exchange information through linear projections and residual fusion:

$$H_g^{(i)} = \tilde{H}_g^{(i)} + (\tilde{H}_s^{(i)}W_s + b_s), \quad H_s^{(i)} = \tilde{H}_s^{(i)} + (\tilde{H}_g^{(i)}W_g + b_g), \quad (43)$$

where $\tilde{H}_g^{(i)}$ and $\tilde{H}_s^{(i)}$ denote the intermediate outputs of the global and sparse branches at layer i , and $H_g^{(i)}$ and $H_s^{(i)}$ are the fused representations. The final node embeddings are typically taken from the global branch and fed into the decoder.

Autoregressive decoding under feasibility. At the decoding step t , CaDA concatenates the embedding of the previously selected node $h_{\tau_t}^{(L)}$ with a 5-D state feature vector $\mathbf{s}_t$ and applies a linear projection to obtain the context embedding H_c :

$$H_c = [h_{\tau_t}^{(L)}, \mathbf{s}_t]W_t, \quad \mathbf{s}_t = [c_t^{\text{LH}}, c_t^{\text{BH}}, t_t, \ell_t, o_t] \in \mathbb{R}^5, \quad (44)$$

where c_t^{LH} and c_t^{BH} denote the remaining capacities for linehaul/backhaul, t_t is the current time, ℓ_t is the current route length, and o_t indicates whether an open route exists. The decoder computes compatibility scores u_i over the feasible set $\mathcal{I}_t$, masks infeasible actions with $-\infty$, and applies Softmax function to obtain the action probabilities p_i :

$$u_i = \begin{cases} \xi \cdot \tanh\left(\frac{q_c^\top h_i^{(L)}}{\sqrt{d_k}}\right), & i \in \mathcal{I}_t, \\ -\infty, & \text{otherwise,} \end{cases} \quad (45)$$

$$p_i = \text{Softmax}(u_i), \quad (46)$$

where ξ is the logit clipping constant, d_k is the feature dimension, and q_c is the query vector derived from H_c .

C. Experiment Details

C.1. Experiment Setup

Baselines. *Traditional Solvers:* We benchmark against two widely used classical VRP solvers. PyVRP (Wouda et al., 2024), built on top of HGS-CVRP (Vidal, 2022), serves as a strong state-of-the-art heuristic baseline, while Google OR-Tools (Perron & Furnon, 2023) provides a robust industrial reference. Both solvers run on a single CPU core per instance

Table 6. Effect of the temperature parameter α on PoMtVRS across 16 VRPs.

α	0.001	0.003	0.005	0.01	0.03	0.05	0.08	0.1	1.0
Avg.Obj	11.489	11.495	11.493	11.491	11.482	11.483	39.180	39.180	20.780
Avg.Gap(%)	1.450	1.527	1.513	1.496	1.405	1.413	258.950	258.950	78.010

with standard time limits that scale with problem size (e.g., 10s for VRP50 and 20s for VRP100). We parallelize traditional solvers across 16 CPU cores as in (Zhou et al., 2024). *Neural Solvers:* We compare against representative multi-task neural VRP solvers spanning unified training, expert specialization, and constraint-aware architectures. Specifically, we include MTPOMO (Liu et al., 2024), MVMoE (Zhou et al., 2024), and the RouteFinder family (Berto et al., 2025), i.e., RF-POMO, RF-MoE, and RF-TE, which provide strong unified-model baselines under consistent training recipes. We further consider CaDA (Li et al., 2025) (a constraint-aware dual-attention mechanism) and MoSES (Pan et al., 2025b) (a mixture-of-specialized-experts solver for compositional VRP variants). For each method, we train and evaluate the models under a unified hardware setup, following the same hyperparameter configurations and problem settings as in RouteFinder (Berto et al., 2025).

PoMtVRS Architecture. To demonstrate the plug-and-play versatility of PoMtVRS, we integrate it into two representative backbones: (i) MVMoE, a typical POMO-based method for multi-task settings, and (ii) CaDA, a recent state-of-the-art approach built on a unified VRP environment. The resulting variants are denoted as PoMtVRS(MVMoE) and PoMtVRS(CaDA), respectively. For fair comparison, all models are trained under the same unified VRP environment with identical hyperparameter configurations, using the same datasets and the same number of training epochs.

Training. We train models on two instance sizes ($N \in \{50, 100\}$), following the standard protocol in prior work (Berto et al., 2025; Li et al., 2025; Pan et al., 2025b). Each model is trained for 300 epochs containing 100,000 VRP instances, where all attributes are generated online to ensure dynamic diversity. We use the Adam optimizer (Kingma & Ba, 2015) with a learning rate of 3×10^{-4} , weight decay of 1×10^{-6} , and batch size of 256. The learning rate is decayed by a factor of 0.1 at epochs 270 and 295. Training is performed over all available variants using the same data budget across all neural methods. For MVMoE and CaDA, we train separate models from scratch for VRP50 and VRP100 with their original model architectures (Appendix B) and the hyperparameters in Appendix C.3. All experiments are conducted on NVIDIA Tesla V100-32GB GPU.

Evaluation. We evaluate using the same test dataset as in (Berto et al., 2025; Li et al., 2025; Pan et al., 2025b). For each VRP variant and each problem size, we use a standardized test set of 1K randomly generated instances. For all neural solvers, we apply a greedy rollout strategy with $\times 8$ augmentation (Kool et al., 2019): each instance is transformed into eight equivalent augmented instances, and we report the best solution among the eight rollouts. We report the average objective and the optimality gap over the 1K test instances, as well as the inference time. The gap is computed with respect to the best heuristic solver results (marked by "*" in Table 1).

C.2. Choice of Temperature Parameter α

In the PO framework, the temperature parameter α is derived from the entropy-regularized objective and controls the exploration-exploitation trade-off. Specifically, a larger α encourages more exploration, whereas a smaller α places more emphasis on exploitation. Prior work (Pan et al., 2025a) provides a practical guideline that α should be chosen according to the solver’s inherent exploration capacity and the problem family. For VRPs, effective values are typically found at the $10^{-2} \sim 10^{-3}$ scale. Following this guideline, we conducted a grid search over α . Specifically, during model selection, we first considered the candidate set $\{0.005, 0.01, 0.05, 0.1, 0.5, 1.0, 2.0\}$. For PoMtVRS, we then performed a finer search around the empirically effective range and additionally evaluated $\{0.001, 0.003, 0.005, 0.01, 0.03, 0.05, 0.08, 0.1, 1\}$, with the results reported in Table 6. The results show that $\alpha = 0.03$ achieves the best performance in PoMtVRS. This suggests that the sensitivity of α is structured rather than arbitrary. Once the problem class and the backbone are fixed, the effective range becomes relatively clear.

C.3. Hyperparameters

Table 7 summarizes the hyperparameter settings used in our main experiments for two representative multi-task backbones (MVMoE and CaDA) and their corresponding PoMtVRS instantiations. To ensure a fair comparison, all methods are trained and evaluated under the same training protocol, including identical optimizer and learning-rate scheduling, batch size,

Table 7. Experiment Hyperparameters.

Hyperparameter	Value	Hyperparameter	Value	Hyperparameter	Value
MVMoE Model		CaDA Model		Training	
Embedding dimension d_h	128	Embedding dimension d_h	128	Batch size	256
Number of attention heads M_h	8	Number of attention heads M_h	8	Optimizer	AdamW
Number of encoder layers L_e	6	Number of encoder layers L_e	6	Learning rate (LR)	$3e^{-4}$
Number of decoder layers L_d	1	Number of decoder layers L_d	1	Weight decay	$1e^{-6}$
Tanh clipping ξ	10.0	Tanh clipping ξ	10.0	LR scheduler	MultiStepLR
Feedforward hidden dimension d_f	512	Feedforward hidden dimension d_f	512	LR milestones	[270, 295]
Routing level	node	Feedforward activation	SwiGLU	LR gamma	0.1
Routing method	Input choice	Feedforward structure	Gated MLP	Train data per epoch	100,000
Normalization	Instance	Normalization	RMSNorm	Training epochs	300
Number of experts	4	Sparse	Top-k	Number of tasks used for training	16
Top-k	2	Top-k	N/2	Gradient clip value	1.0

number of epochs and training samples per epoch, the number of tasks, and gradient clipping. The core model capacity is also matched across methods (e.g., embedding dimension, attention heads, encoder depth, and feed-forward width), while only method-specific components and mechanisms (e.g., MoE or gated modules with Top-k selection, normalization and sparsity choices) follow their original designs, so that the observed differences can be attributed to the proposed approach rather than disparate training or capacity settings.

C.4. Ablation Study Details

In the main paper, we report the averaged ablation results over 16 VRP variants in Table 2 of Section 5.2. Table 8 further provides the detailed results on two problem scales (VRP50 and VRP100), including the objective values and average optimality gaps of each ablation variant and baseline across all tasks. These results offer a more fine-grained view of how PO and the PGB contribute to the final performance.

Effectiveness of the Preference-gated Block. In terms of the average optimality gap, the full PoMtVRS(CaDA) consistently achieves the best performance on both scales. It reduces the average gap of the CaDA backbone from 1.721% to 1.405% on VRP50 and from 2.844% to 2.205% on VRP100. When the entire PGB is removed while PO is retained (w.o. block), the average gap increases to 1.579% on VRP50 and 2.529% on VRP100, showing a clear degradation relative to the full model. This confirms that PGB is a key component for improving decoder representations under heterogeneous routing constraints. We further examine the internal components of PGB. Removing the gating mechanism (w.o. Gate) worsens the average gap to 1.437% on VRP50 and 2.245% on VRP100, indicating that preference-conditioned feature modulation is important for effective multi-task adaptation. Similarly, removing the state attribute mapping branch (w.o. Map) leads to average gaps of 1.424% and 2.323% on VRP50 and VRP100, respectively, which are also inferior to the full model. This suggests that state attribute mapping provides useful task- and state-aware signals for guiding the gated representation refinement, especially on larger instances where constraint interactions become more complex. Overall, the degradation caused by removing either the whole block or its internal components verifies that the improvement of PGB does not simply come from adding parameters, but from its structured integration of preference information, gating modulation, and state-aware mapping.

Reinforcement Learning (RL) vs. Preference Optimization (PO). With the same CaDA backbone, replacing standard RL with PO already brings consistent improvements. Specifically, comparing CaDA with w.o. block, the average gap decreases from 1.721% to 1.579% on VRP50 and from 2.844% to 2.529% on VRP100. This indicates that the relative preference signal provides a more stable and better-aligned learning objective than absolute-return optimization in multi-task VRP settings, where reward scales may vary across different variants. Meanwhile, keeping PGB but removing PO (w.o. PO) still improves over CaDA, achieving average gaps of 1.621% on VRP50 and 2.453% on VRP100. However, it remains clearly worse than the full PoMtVRS(CaDA), whose gaps are 1.405% and 2.205% on the two scales. These results show that PGB and PO are complementary: PGB enhances the decoder representation, while PO provides a more suitable optimization signal to fully exploit this representation enhancement.

Preference-gated Block vs. ReLD-based Architecture. To rule out the hypothesis that arbitrary architectural tweaks are sufficient, we compare against ReLD-style modifications. Under the ReLD configuration, introducing PO improves the average gap from 1.714% to 1.438% on VRP50 and from 2.694% to 2.529% on VRP100, confirming the effectiveness of PO beyond the proposed architecture. However, w.o. PO-ReLD is still inferior to PoMtVRS(CaDA), especially on VRP100

Table 8. Ablation and comparison studies on the proposed PoMtVRS across 16 VRPs.

N=50	CaDA		w.o. block		w.o. Gate		w.o. Map		w.o. PO		w.o. RL-Reld		w.o. PO-Reld		PoMtVRS(CaDA)	
	Obj.	Gap	Obj.	Gap	Obj.	Gap	Obj.	Gap	Obj.	Gap	Obj.	Gap	Obj.	Gap	Obj.	Gap
CVRP	10.495	1.186%	10.479	1.032%	10.462	0.875%	10.458	0.833%	10.477	1.012%	10.491	1.142%	10.461	0.861%	10.459	0.843%
OVRP	6.670	2.467%	6.657	2.269%	6.644	2.075%	6.639	2.003%	6.665	2.396%	6.672	2.504%	6.643	2.054%	6.640	2.010%
VRPB	9.960	2.798%	9.934	2.529%	9.902	2.206%	9.902	2.205%	9.933	2.525%	9.958	2.783%	9.901	2.196%	9.899	2.171%
VRPBL	10.548	3.508%	10.529	3.308%	10.496	2.985%	10.490	2.937%	10.521	3.236%	10.547	3.488%	10.495	2.969%	10.494	2.966%
VRPBTW	18.504	1.140%	18.493	1.073%	18.480	1.012%	18.486	1.040%	18.503	1.135%	18.507	1.160%	18.485	1.033%	18.478	0.995%
OVRPB	7.047	2.129%	7.034	1.940%	7.020	1.786%	7.022	1.770%	7.044	2.092%	7.049	2.162%	7.022	1.773%	7.021	1.755%
OVRPBLTW	11.761	0.778%	11.751	0.696%	11.745	0.646%	11.746	0.656%	11.757	0.747%	11.758	0.757%	11.745	0.644%	11.744	0.637%
OVRPL	6.669	2.451%	6.656	2.261%	6.644	2.072%	6.641	2.024%	6.666	2.406%	6.674	2.532%	6.642	2.051%	6.639	2.004%
VRPTW	16.276	1.522%	16.261	1.428%	16.249	1.349%	16.246	1.338%	16.268	1.470%	16.278	1.533%	16.249	1.346%	16.236	1.272%
VRPL	10.730	1.330%	10.715	1.193%	10.698	1.032%	10.696	1.010%	10.712	1.161%	10.731	1.334%	10.698	1.034%	10.698	1.028%
OVRPTW	10.619	1.013%	10.604	0.876%	10.595	0.786%	10.596	0.802%	10.608	0.909%	10.613	0.957%	10.597	0.806%	10.593	0.775%
VRPBLTW	18.852	1.402%	18.843	1.342%	18.821	1.228%	18.828	1.267%	18.842	1.346%	18.843	1.356%	18.837	1.313%	18.814	1.194%
VRPLTW	16.670	1.890%	16.652	1.783%	16.630	1.764%	16.634	1.672%	16.650	1.771%	16.664	1.858%	16.636	1.684%	16.634	1.664%
OVRPBL	7.049	2.146%	7.037	1.962%	7.022	1.763%	7.023	1.770%	7.044	2.078%	7.050	2.154%	7.024	1.784%	7.022	1.755%
OVRPBTW	11.759	0.757%	11.752	0.696%	11.745	0.643%	11.746	0.653%	11.757	0.742%	11.759	0.758%	11.746	0.649%	11.744	0.634%
OVRPLTW	10.620	1.021%	10.604	0.876%	10.593	0.775%	10.596	0.796%	10.608	0.908%	10.612	0.951%	10.597	0.807%	10.593	0.776%
Avg.	11.514	1.721%	11.500	1.579%	11.484	1.437%	11.484	1.423%	11.503	1.621%	11.513	1.714%	11.486	1.438%	11.482	1.405%

N=100	CaDA		w.o. block		w.o. Gate		w.o. Map		w.o. PO		w.o. RL-Reld		w.o. PO-Reld		PoMtVRS(CaDA)	
	Obj.	Gap	Obj.	Gap	Obj.	Gap	Obj.	Gap	Obj.	Gap	Obj.	Gap	Obj.	Gap	Obj.	Gap
CVRP	15.874	1.601%	15.818	1.244%	15.769	0.998%	15.819	1.052%	15.804	1.158%	15.897	1.552%	15.838	1.179%	15.768	0.926%
OVRP	10.127	4.103%	10.076	3.581%	10.043	3.257%	10.066	3.319%	10.057	3.388%	10.133	4.000%	10.083	3.493%	10.034	3.155%
VRPB	14.963	4.080%	14.886	3.547%	14.807	2.993%	14.842	3.106%	14.854	3.336%	14.950	3.861%	14.877	3.372%	14.806	2.993%
VRPBL	15.530	5.029%	15.468	4.610%	15.362	3.891%	15.440	4.108%	15.407	4.218%	15.574	5.005%	15.464	4.298%	15.351	3.832%
VRPBTW	30.072	2.044%	30.029	1.889%	29.990	1.760%	30.054	1.808%	30.042	1.945%	30.072	1.868%	30.107	1.991%	29.981	1.729%
OVRPB	10.761	4.083%	10.700	3.501%	10.660	3.121%	10.680	3.227%	10.686	3.371%	10.748	3.875%	10.704	3.457%	10.657	3.093%
OVRPBLTW	19.441	1.468%	19.421	1.360%	19.397	1.238%	19.428	1.290%	19.425	1.390%	19.436	1.324%	19.463	1.477%	19.395	1.224%
OVRPL	10.127	4.100%	10.078	3.599%	10.042	3.244%	10.067	3.322%	10.056	3.376%	10.133	3.996%	10.083	3.492%	10.033	3.146%
VRPTW	26.083	2.579%	26.020	2.327%	25.987	2.192%	26.049	2.255%	26.043	2.419%	26.062	2.304%	26.109	2.492%	25.976	2.151%
VRPL	16.061	1.875%	16.009	1.551%	15.954	1.200%	16.025	1.313%	15.988	1.418%	16.111	1.852%	16.056	1.504%	15.951	1.188%
OVRPTW	17.234	1.793%	17.206	1.625%	17.180	1.472%	17.206	1.541%	17.214	1.677%	17.217	1.605%	17.242	1.752%	17.175	1.443%
VRPBLTW	30.534	2.407%	30.506	2.306%	30.447	2.107%	30.527	2.134%	30.497	2.288%	30.553	2.223%	30.600	2.388%	30.429	2.048%
VRPLTW	26.534	2.977%	26.502	2.845%	26.439	2.600%	26.514	2.669%	26.494	2.824%	26.563	2.854%	26.569	2.877%	26.433	2.581%
OVRPBL	10.762	4.093%	10.700	3.498%	10.662	3.135%	10.677	3.195%	10.687	3.382%	10.748	3.880%	10.705	3.465%	10.659	3.110%
OVRPBTW	19.441	1.472%	19.420	1.358%	19.397	1.241%	19.427	1.282%	19.425	1.388%	19.433	1.308%	19.463	1.479%	19.394	1.222%
OVRPLTW	17.234	1.793%	17.205	1.623%	17.180	1.471%	17.207	1.543%	17.212	1.666%	17.216	1.597%	17.242	1.749%	17.174	1.440%
Avg.	18.173	2.844%	18.128	2.529%	18.082	2.245%	18.127	2.323%	18.118	2.453%	18.178	2.694%	18.163	2.529%	18.076	2.205%

where the gap remains 2.529% compared with 2.205% of the full model. This demonstrates that the performance gain is not merely due to adopting an alternative decoder modification but mainly stems from the targeted coupling between PO and the proposed PGB. Overall, the detailed ablation results confirm that PO, PGB, the gating mechanism, and the state attribute mapping branch all contribute to the final performance, and their joint design leads to more stable optimization, stronger representation refinement, and better generalization across diverse VRP variants.

C.5. Generalization on Real-World Instances

We further evaluate PoMtVRS(CaDA) on the CVRPLIB instances from the X-set, following the standard protocol adopted by recent unified VRP solvers (e.g., MVMoE, RouteFinder, CaDA, and MoSES []). This benchmark contains 100 instances ranging from 101 to 1,001 nodes. While Table 3 in the main text reports the size-binned average gaps, Table 9 provides instance-wise comparisons on every instance for all methods. For clarity, we partition the test set into three subsets by instance size, consistent with prior work: $N < 251$, $251 \leq N < 501$, and $501 < N \leq 1001$.

As summarized from Table 9, PoMtVRS(CaDA) consistently outperforms both the CaDA baseline and the strong competitor MoSES(CaDA) across all three subsets. Specifically, on $N < 251$, PoMtVRS(CaDA) reduces the average gap from 4.772% (CaDA) to 4.561%, also improving over 4.724% by MoSES(CaDA). On $251 \leq N < 501$, PoMtVRS(CaDA) achieves 7.865%, substantially better than 8.889% (CaDA) and also better than 7.948% (MoSES (CaDA)). On the most challenging subset $501 < N \leq 1001$, PoMtVRS(CaDA) reaches 12.651%, improving upon 14.199% (CaDA) and 12.814% (MoSES(CaDA)). Overall, across all 100 instances, PoMtVRS(CaDA) lowers the average gap from 9.230% (CaDA) to 8.326%, and surpasses 8.441% from MoSES(CaDA). These results show that, under identical evaluation budgets and inference settings, PoMtVRS(CaDA) achieves consistent improvements across problem scales, with larger gains on medium- and large-scale instances, indicating stronger robustness on realistic benchmarks and improved scalability. In addition, relative to other multi-task baselines, PoMtVRS(CaDA) attains a leading or highly competitive average gap, which supports its effectiveness on the real-world instances.

PoMtVRS: Preference-Optimized Multi-Task Vehicle Routing Solver with Preference Gating

Table 9. Experimental results on CVRPLIB instances from the X set.

Instance	Set-X	MTPOMO				MVMoE				RF-MoE				RF-POMO				RF-TE				CaDA				MoSES(CaDA)				PoMtVRS(CaDA)			
		Obj.	Gap	Time	Obj.	Gap	Time	Obj.	Gap	Time	Obj.	Gap	Time	Obj.	Gap	Time	Obj.	Gap	Time	Obj.	Gap	Time	Obj.	Gap	Time	Obj.	Gap	Time					
X-n101-k25	27591	29470	6.810%	0.4s	29076	5.382%	0.5s	28934	4.868%	0.5s	29090	5.433%	0.3s	29048	5.281%	0.4s	28944	4.904%	0.5s	29110	5.505%	0.8s	28941	4.893%	0.6s	28941	4.893%	0.6s					
X-n106-k14	26362	28029	6.323%	0.3s	27443	4.101%	0.5s	27292	3.528%	0.6s	27378	3.854%	0.3s	27159	3.023%	0.4s	27042	2.579%	0.3s	27051	2.614%	0.8s	27051	2.614%	0.8s	27051	2.614%	0.8s					
X-n110-k13	14971	15100	0.862%	0.3s	15327	2.378%	0.5s	15260	1.930%	0.5s	15519	3.660%	0.3s	15314	2.291%	0.4s	15229	1.723%	0.3s	15332	2.411%	0.8s	15332	2.411%	0.8s	15332	2.411%	0.8s					
X-n115-k10	12747	13433	5.382%	0.4s	13475	5.711%	0.6s	13638	6.990%	0.5s	13263	4.048%	0.3s	13338	4.636%	0.4s	13060	2.455%	0.4s	13085	2.652%	0.8s	13085	2.652%	0.8s	13085	2.652%	0.8s					
X-n120-k6	13332	14051	5.393%	0.3s	13782	3.373%	0.6s	13908	4.320%	0.6s	14067	5.468%	0.4s	13765	3.248%	0.4s	13678	2.595%	0.3s	13619	2.133%	0.8s	13619	2.133%	0.8s	13619	2.133%	0.8s					
X-n125-k30	55539	59015	6.259%	0.4s	58200	4.791%	0.7s	58587	5.488%	0.6s	58770	5.818%	0.4s	58570	5.457%	0.4s	57748	3.977%	0.4s	57620	3.747%	1.0s	57404	3.358%	0.7s	57404	3.358%	0.7s					
X-n129-k18	28940	30176	4.271%	0.4s	29334	1.361%	0.6s	30039	3.798%	0.6s	29645	2.436%	0.4s	29457	1.786%	0.5s	29500	1.935%	0.4s	29620	2.350%	0.9s	29630	2.384%	0.8s	29630	2.384%	0.8s					
X-n134-k13	10916	11707	7.246%	0.4s	11462	5.002%	0.6s	11439	4.791%	0.6s	11463	5.011%	0.4s	11624	6.486%	0.4s	11652	6.742%	0.4s	11573	6.019%	1.0s	11241	2.977%	0.8s	11241	2.977%	0.8s					
X-n139-k10	13590	14058	3.444%	0.4s	14099	3.745%	0.6s	13917	2.406%	0.6s	13945	2.612%	0.4s	13812	1.634%	0.4s	13940	2.575%	0.4s	13877	2.112%	0.9s	13806	1.589%	0.8s	13806	1.589%	0.8s					
X-n143-k7	15700	16626	5.898%	0.4s	16349	4.134%	0.6s	16655	6.083%	0.6s	16603	5.752%	0.5s	16257	3.548%	0.4s	16189	3.115%	0.4s	15980	1.783%	0.9s	16085	2.452%	0.8s	16085	2.452%	0.8s					
X-n148-k46	43448	46648	7.365%	0.5s	45893	5.627%	0.8s	46542	7.121%	0.8s	46082	6.062%	0.5s	45026	3.632%	0.6s	45606	4.967%	0.6s	45600	4.953%	1.0s	45476	4.668%	0.9s	45476	4.668%	0.9s					
X-n153-k22	21220	23514	10.811%	0.5s	23661	11.503%	0.7s	23906	12.658%	0.7s	22991	8.346%	0.5s	23478	10.641%	0.6s	23142	9.057%	0.5s	23310	9.849%	1.0s	22897	7.902%	0.8s	22897	7.902%	0.8s					
X-n157-k13	16876	17922	6.198%	0.5s	17439	3.336%	0.7s	17801	5.481%	0.8s	17536	3.911%	0.5s	17315	2.601%	0.5s	17295	2.483%	0.5s	17317	2.613%	1.0s	17168	1.730%	0.9s	17168	1.730%	0.9s					
X-n162-k11	14138	14616	3.381%	0.5s	14705	4.010%	0.7s	14524	2.730%	0.7s	14663	3.713%	0.5s	14664	3.720%	0.5s	14704	4.003%	0.5s	14677	3.812%	1.0s	14517	2.681%	0.9s	14517	2.681%	0.9s					
X-n167-k10	20557	21662	5.375%	0.5s	21504	4.607%	0.7s	21481	4.495%	0.7s	21410	4.149%	0.5s	21425	4.222%	0.5s	21078	2.534%	0.5s	21384	4.023%	1.0s	21663	5.800%	0.9s	21663	5.800%	0.9s					
X-n172-k51	45607	48960	7.352%	0.6s	47883	4.990%	0.9s	49726	9.032%	1.0s	48412	6.150%	0.6s	48162	5.602%	0.7s	48198	5.681%	0.6s	48145	5.565%	1.0s	47933	5.180%	0.9s	47933	5.180%	0.9s					
X-n176-k26	47812	51989	8.736%	0.5s	52117	9.004%	0.8s	53626	12.160%	0.9s	52347	9.485%	0.6s	51801	7.116%	0.6s	51120	6.819%	0.6s	51612	7.948%	1.0s	52194	9.165%	0.9s	52194	9.165%	0.9s					
X-n181-k23	25569	26572	3.923%	0.6s	26456	3.469%	0.8s	26154	1.402%	1.0s	26544	3.813%	0.6s	26097	2.065%	0.6s	26262	2.710%	0.6s	26143	2.245%	1.0s	26035	1.822%	0.9s	26035	1.822%	0.9s					
X-n186-k15	24145	25236	4.519%	0.5s	25151	4.166%	0.8s	25140	4.121%	1.0s	25238	4.527%	0.5s	25153	4.175%	0.6s	25345	4.970%	0.6s	25246	4.560%	1.0s	25191	4.332%	0.9s	25191	4.332%	0.9s					
X-n190-k8	16980	18369	8.180%	0.5s	19078	12.356%	0.9s	18217	7.285%	0.9s	18696	10.106%	0.6s	17871	5.247%	0.6s	17882	5.312%	0.6s	17569	3.469%	1.0s	17818	4.935%	0.9s	17818	4.935%	0.9s					
X-n195-k51	44225	48310	9.237%	0.7s	46974	6.216%	1.0s	48965	10.718%	1.0s	47479	7.358%	0.7s	47396	7.170%	0.7s	47396	7.170%	0.7s	47479	7.358%	1.0s	47081	6.458%	0.9s	47081	6.458%	0.9s					
X-n200-k36	58578	62041	5.912%	0.6s	61627	5.205%	0.9s	61696	5.323%	0.9s	61662	5.265%	0.6s	61139	4.372%	0.7s	61010	4.152%	0.8s	61089	4.287%	2.0s	61314	4.670%	0.9s	61314	4.670%	0.9s					
X-n204-k19	58578	20652	5.556%	0.6s	20584	5.208%	0.9s	20466	4.605%	1.0s	20730	5.955%	0.6s	20531	4.937%	0.6s	20735	5.980%	0.6s	20420	4.370%	1.0s	20397	4.252%	0.9s	20397	4.252%	0.9s					
X-n209-k16	30656	32333	5.470%	0.6s	32358	5.552%	0.9s	32145	4.857%	0.9s	32585	6.292%	0.6s	31876	3.980%	0.6s	32184	4.984%	0.6s	32053	4.557%	1.0s	32012	4.423%	0.9s	32012	4.423%	0.9s					
X-n214-k11	10856	11699	7.765%	0.6s	11597	6.826%	0.9s	11534	6.245%	0.9s	11638	7.203%	0.6s	11668	7.480%	0.6s	11748	8.217%	0.6s	11716	7.922%	1.0s	11741	8.152%	0.9s	11741	8.152%	0.9s					
X-n219-k73	117595	121980	3.729%	0.8s	12434	5.816%	1.0s	121627	3.429%	1.0s	123500	5.021%	0.8s	120344	2.338%	0.8s	120011	2.055%	0.8s	119710	1.799%	2.0s	120837	2.757%	1.2s	120837	2.757%	1.2s					
X-n223-k34	40437	43381	7.290%	0.7s	42694	5.582%	1.0s	43097	6.578%	1.0s	42601	5.512%	0.7s	42251	4.486%	0.7s	42273	4.540%	0.7s	42128	4.182%	2.0s	42305	4.695%	1.2s	42305	4.695%	1.2s					
X-n228-k23	25742	28523	10.803%	0.7s	28033	8.900%	1.0s	29590	14.948%	1.0s	28212	9.255%	0.8s	28699	11.487%	0.8s	27821	8.076%	0.7s	27724	7.699%	1.0s	27733	7.734%	1.0s	27733	7.734%	1.0s					
X-n233-k16	19230	20644	7.353%	0.7s	20656	7.415%	1.0s	20507	6.644%	1.0s	20427	6.225%	0.7s	20761	7.962%	0.7s	20285	5.486%	0.9s	20623	7.244%	1.0s	20433	6.256%	1.2s	20433	6.256%	1.2s					
X-n237-k14	27042	30047	11.125%	0.7s	29772	10.095%	1.0s	29514	9.141%	1.0s	30044	11.249%	0.7s	29595	9.441%	0.7s	30262	11.981%	0.7s	29518	9.156%	1.0s	29883	10.503%	1.0s	29883	10.503%	1.0s					
X-n242-k48	82751	88179	6.559%	0.8s	87497	5.735%	1.0s	87832	6.140%	1.0s	87029	5.170%	0.8s	85704	3.569%	0.9s	85813	3.700%	0.8s	85643	3.495%	2.0s	86726	4.803%	1.2s	86726	4.803%	1.2s					
X-n247-k50	37274	41610	11.633%	0.8s	40973	9.924%	1.0s	43153	15.772%	1.0s	41120	10.318%	0.8s	40642	9.036%	0.9s	39918	7.093%	0.8s	40736	9.288%	2.0s	41510	11.364%	1.2s	41510	11.364%	1.2s					
Avg. Gap ($N \leq 251$)		6.566%				5.829%				6.754%				5.905%				5.061%				4.772%				4.724%				4.561%			
X-n251-k28	38684	41211	6.532%	0.7s	41330	6.340%	1.0s	40691	5.188%	1.0s	40811	5.498%	0.8s	40127	3.730%	0.8s	40359	4.330%	0.8s	40290	4.152%	2.0s	40385	4.397%	1.2s	40385	4.397%	1.2s					
X-n256-k16	18839	20400	8.286%	0.7s	20559	9.180%	1.0s	20015	6.242%	1.0s	20238	7.426%	0.7s	19994	6.131%	0.8s	20372	8.137%	0.7s	20068	6.524%	2.0s	19827	5.244%	1.2s	19827	5.244%	1.2s					
X-n261-k13	26558	28741	8.220%	0.7s	28524	7.403%	1.0s	28203	6.194%	1.0s	28525	7.406%	0.8s	28510	7.350%	0.8s	28833	8.566%	1.0s	28577	7.602%	2.0s	29060	9.420%	1.5s	29060	9.420%	1.5s					
X-n266-k58	75478	84747	12.108%	0.9s	82048	8.705%	1.0s	81135	7.495%	1.0s	81053	7.386%	0.9s	79832	5.769%	0.9s	80115	6.144%	0.9s	80036	6.039%	2.0s	79792	5.715%	1.5s	79792	5.715%	1.5s					
X-n270-k35	35291	38146	8.090%	0.9s	38333	8.620%	1.0s																										

D. Efficient Activate Search with LoRA for Performance

Efficient active search (EAS) for better solution quality. To further improve the quality of solutions obtained during testing of PoMtVRS(CaDA) on synthetic datasets, this work adopts an efficient active search (added-layer update strategy). Following the EAS-Lay paradigm, we augment the pretrained decoder with a lightweight, instance-specific residual module while keeping all original model parameters fixed during search. The added module is optimized per batch through gradient-based updates, enabling fast instance-level adaptation under a fixed inference budget. For each test instance, an instance-specific residual transform is applied to the decoder multi-head attention output M_t^H :

$$\tilde{M}_t^H = M_t^H + \left(\text{ReLU}(M_t^H W_1 + b_1) W_2 + b_2 \right), \quad (47)$$

where W_1, b_1, W_2 and b_2 denote the trainable parameters during search. Given the input h , the added layer applies two linear transformations with a ReLU nonlinearity in between. W_1, W_2 and their corresponding biases b_1, b_2 are optimized via gradient descent throughout the search. To ensure that the added layer does not perturb the original model output at the beginning of search, W_2 and b_2 are initialized to zero.

Low-rank adaptation (LoRA) for faster search. While EAS significantly improves solution quality through iterative online gradient updates, it also introduces considerable runtime overhead due to repeated backpropagation and updates of high-dimensional decoder projections. To mitigate the search cost while preserving the solution-quality improvements from EAS, we incorporate low-rank adaptation (LoRA), which shifts the online optimization from updating full high-dimensional weights to learning small, low-rank updates. This approach substantially reduces the number of trainable parameters and the optimization complexity, thereby lowering per-iteration update costs and improving efficiency. For a weight matrix W , LoRA parameterizes a low-rank update as:

$$W' = W + \frac{\alpha}{r} B A, \quad (48)$$

where $A \in \mathbb{R}^{r \times d_{\text{in}}}$ and $B \in \mathbb{R}^{d_{\text{out}} \times r}$ are trainable low-rank factors, while the original weight matrix W is kept frozen. During the search, we update only the EAS-Lay parameters and the LoRA parameters (A, B), while freezing all other weights. This reduces the parameter space for online adaptation, leading to effective improvements in fewer iterations, making early stopping more likely and significantly shortening the overall search time with comparable performance gains.

Experiments and results. Table 10 reports results on 1K test instances across 16 VRP variants at two scales ($N = 50$ and $N = 100$). Without active search, PoMtVRS(CaDA) already yields a consistent improvement over the CaDA backbone across all variants and both scales, indicating that the preference-driven augmentation generalizes well under diverse constraint combinations. With EAS, both CaDA and PoMtVRS benefit from online refinement, but PoMtVRS(CaDA)-EAS remains consistently stronger and achieves the best overall solution quality among learning-based solvers, exhibiting smaller gaps on a broader range of variants and maintaining its advantage at the larger scale ($N = 100$), where search becomes more challenging. Finally, EAS-LoRA delivers a substantially better accuracy-efficiency balance. It preserves most of the gains of full EAS while significantly reducing the search time across variants, making it a practical default for deployment. In contrast, full EAS provides the strongest performance upper bound when runtime is less constrained.

Table 10. Experimenta results on 1K test instances across 16 VRPs.

Solver		N = 50			N = 100			Solver		N = 50			N = 100		
		Obj.	Gap	Time	Obj.	Gap	Time			Obj.	Gap	Time	Obj.	Gap	Time
CVRP	HGS-PyVRP	10.372	*	10.4m	15.628	*	20.8m	HGS-PyVRP	16.031	*	10.4m	25.423	*	20.8m	
	OR-Tools	10.572	1.907%	10.4m	16.280	4.178%	20.8m	OR-Tools	16.089	0.347%	10.4m	25.814	1.506%	20.8m	
	CaDA	10.495	1.186%	3s	15.874	1.601%	13s	CaDA	16.276	1.522%	3s	26.083	2.579%	13s	
	CaDA-EAS	10.404	0.307%	8.3m	15.701	0.488%	1.09h	CaDA-EAS	16.163	0.815%	9.2m	25.888	1.818%	1.16h	
	PoMtVRS(CaDA)	10.459	0.843%	4s	15.768	0.926%	17s	PoMtVRS(CaDA)	16.236	1.272%	4s	25.976	2.151%	20s	
	PoMtVRS(CaDA)-EAS	10.387	0.144%	16.4m	15.629	0.035%	1.32h	PoMtVRS(CaDA)-EAS	16.117	0.532%	18.4m	25.763	1.317%	1.38h	
	PoMtVRS(CaDA)-EAS-LoRA	10.400	0.273%	6.4m	15.649	0.161%	1.03h	PoMtVRS(CaDA)-EAS-LoRA	16.140	0.674%	8.0m	25.838	1.609%	39.2m	
OVRP	HGS-PyVRP	6.507	*	10.4m	9.725	*	20.8m	HGS-PyVRP	10.587	*	10.4m	15.766	*	20.8m	
	OR-Tools	6.553	0.686%	10.4m	9.995	2.732%	20.8m	OR-Tools	10.570	2.343%	10.4m	16.466	5.302%	20.8m	
	CaDA	6.670	2.467%	3s	10.127	4.103%	13s	CaDA	10.730	1.330%	3s	16.061	1.875%	13s	
	CaDA-EAS	6.597	1.356%	8.5m	10.025	3.051%	1.17h	CaDA-EAS	10.647	0.545%	8.3m	15.971	1.302%	1.13h	
	PoMtVRS(CaDA)	6.640	2.010%	4s	10.034	3.155%	17s	PoMtVRS(CaDA)	10.698	1.028%	4s	15.951	1.188%	17s	
	PoMtVRS(CaDA)-EAS	6.553	0.700%	17.3m	9.929	2.075%	1.33h	PoMtVRS(CaDA)-EAS	10.625	0.339%	16.5m	15.873	0.690%	1.31h	
	PoMtVRS(CaDA)-EAS-LoRA	6.568	0.920%	7.5m	9.958	2.374%	45.4m	PoMtVRS(CaDA)-EAS-LoRA	10.637	0.456%	8.4m	15.896	0.837%	51.1m	
VRPB	HGS-PyVRP	9.687	*	10.4m	14.377	*	20.8m	HGS-PyVRP	10.510	*	10.4m	16.926	*	20.8m	
	OR-Tools	9.802	1.159%	10.4m	14.933	3.853%	20.8m	OR-Tools	10.519	0.078%	10.4m	17.027	0.583%	20.8m	
	CaDA	9.960	2.798%	3s	14.963	4.080%	13s	CaDA	10.619	1.013%	3s	17.234	1.793%	14s	
	CaDA-EAS	9.843	1.591%	8.3m	14.817	3.055%	1.14h	CaDA-EAS	10.561	0.472%	9.4m	17.131	1.189%	1.24h	
	PoMtVRS(CaDA)	9.899	2.171%	4s	14.806	2.993%	17s	PoMtVRS(CaDA)	10.593	0.775%	4s	17.175	1.443%	20s	
	PoMtVRS(CaDA)-EAS	9.787	1.010%	16.5m	14.683	2.128%	1.30h	PoMtVRS(CaDA)-EAS	10.541	0.288%	19.4m	17.059	0.768%	1.43h	
	PoMtVRS(CaDA)-EAS-LoRA	9.804	1.193%	9.4m	14.714	2.340%	43.3m	PoMtVRS(CaDA)-EAS-LoRA	10.551	0.375%	6.1m	17.086	0.924%	51.3m	
VRPBL	HGS-PyVRP	10.186	*	10.4m	14.779	*	20.8m	HGS-PyVRP	18.361	*	10.4m	29.026	*	20.8m	
	OR-Tools	10.331	1.390%	10.4m	15.426	4.338%	20.8m	OR-Tools	18.422	0.332%	10.4m	29.830	2.770%	20.8m	
	CaDA	10.548	3.508%	3s	15.530	5.029%	13s	CaDA	18.852	1.402%	3s	30.534	2.407%	14s	
	CaDA-EAS	10.379	1.859%	8.4m	15.321	3.630%	1.15h	CaDA-EAS	18.714	0.663%	9.3m	30.295	1.610%	1.24h	
	PoMtVRS(CaDA)	10.494	2.966%	4s	15.351	3.832%	18s	PoMtVRS(CaDA)	18.814	1.194%	4s	30.429	2.048%	20s	
	PoMtVRS(CaDA)-EAS	10.320	1.290%	17.2m	15.157	2.517%	1.31h	PoMtVRS(CaDA)-EAS	18.671	0.436%	19.2m	30.088	0.919%	1.41h	
	PoMtVRS(CaDA)-EAS-LoRA	10.354	1.614%	9.0m	15.212	2.891%	51.4m	PoMtVRS(CaDA)-EAS-LoRA	18.701	0.595%	6.4m	30.145	1.107%	59.2m	
VRPBLTW	HGS-PyVRP	18.292	*	10.4m	29.467	*	20.8m	HGS-PyVRP	16.356	*	10.4m	25.757	*	20.8m	
	OR-Tools	18.366	0.383%	10.4m	29.945	1.597%	20.8m	OR-Tools	16.441	0.499%	10.4m	26.259	1.899%	20.8m	
	CaDA	18.504	1.140%	3s	30.072	2.044%	14s	CaDA	16.670	1.890%	3s	26.534	2.977%	14s	
	CaDA-EAS	18.378	0.464%	9.3m	29.846	1.278%	1.23h	CaDA-EAS	16.517	0.971%	9.2m	26.295	2.060%	1.21h	
	PoMtVRS(CaDA)	18.478	0.995%	4s	29.981	1.729%	20s	PoMtVRS(CaDA)	16.634	1.664%	4s	26.433	2.581%	20s	
	PoMtVRS(CaDA)-EAS	18.358	0.352%	19.1m	15.768	0.926%	1.40h	PoMtVRS(CaDA)-EAS	16.461	0.630%	18.5m	26.141	1.462%	1.39h	
	PoMtVRS(CaDA)-EAS-LoRA	18.386	0.502%	5.3m	29.753	0.960%	57.0m	PoMtVRS(CaDA)-EAS-LoRA	16.491	0.811%	8.4m	26.225	1.783%	47.1m	
OVRPB	HGS-PyVRP	6.898	*	10.4m	10.335	*	20.8m	HGS-PyVRP	6.899	*	10.4m	10.335	*	20.8m	
	OR-Tools	6.928	0.412%	10.4m	10.577	2.315%	20.8m	OR-Tools	6.927	0.386%	10.4m	10.582	2.363%	20.8m	
	CaDA	7.047	2.129%	3s	10.761	4.083%	13s	CaDA	7.049	2.146%	3s	10.762	4.093%	13s	
	CaDA-EAS	6.956	0.824%	8.5m	10.634	2.859%	1.10h	CaDA-EAS	6.982	1.182%	8.5m	10.593	2.467%	1.10h	
	PoMtVRS(CaDA)	7.021	1.755%	4s	10.657	3.093%	17s	PoMtVRS(CaDA)	7.022	1.755%	4s	10.659	3.110%	18s	
	PoMtVRS(CaDA)-EAS	6.939	0.581%	17.4m	10.508	1.655%	1.33h	PoMtVRS(CaDA)-EAS	6.941	0.591%	17.5m	10.480	1.381%	1.33h	
	PoMtVRS(CaDA)-EAS-LoRA	6.952	0.763%	8.0m	10.542	1.977%	57.2m	PoMtVRS(CaDA)-EAS-LoRA	6.954	0.772%	8.1m	10.496	1.535%	1.00h	
OVRPBLTW	HGS-PyVRP	11.668	*	10.4m	19.156	*	20.8m	HGS-PyVRP	11.669	*	10.4m	19.156	*	20.8m	
	OR-Tools	11.681	0.106%	10.4m	19.305	0.767%	20.8m	OR-Tools	11.682	0.109%	10.4m	19.303	0.757%	20.8m	
	CaDA	11.761	0.778%	3s	19.441	1.468%	14s	CaDA	11.759	0.757%	3s	19.441	1.472%	14s	
	CaDA-EAS	11.711	0.362%	10.0m	19.281	0.639%	1.20h	CaDA-EAS	11.709	0.337%	10.0m	19.320	0.842%	1.19h	
	PoMtVRS(CaDA)	11.744	0.637%	5s	19.395	1.224%	20s	PoMtVRS(CaDA)	11.744	0.634%	5s	19.394	1.222%	20s	
	PoMtVRS(CaDA)-EAS	11.694	0.213%	20.2m	19.238	0.418%	1.46h	PoMtVRS(CaDA)-EAS	11.695	0.219%	20.2m	19.247	0.465%	1.46h	
	PoMtVRS(CaDA)-EAS-LoRA	11.703	0.292%	5.1m	19.251	0.487%	59.3m	PoMtVRS(CaDA)-EAS-LoRA	11.703	0.285%	6.2m	19.268	0.574%	59.5m	
OVRPL	HGS-PyVRP	6.507	*	10.4m	9.724	*	20.8m	HGS-PyVRP	10.510	*	10.4m	16.926	*	20.8m	
	OR-Tools	6.552	0.668%	10.4m	10.001	2.791%	20.8m	OR-Tools	10.497	0.114%	10.4m	17.023	0.728%	20.8m	
	CaDA	6.669	2.451%	3s	10.127	4.100%	13s	CaDA	10.620	1.021%	3s	17.234	1.793%	14s	
	CaDA-EAS	6.583	1.145%	8.5m	10.018	2.984%	1.17h	CaDA-EAS	10.563	0.491%	9.4m	17.128	1.173%	1.25h	
	PoMtVRS(CaDA)	6.639	2.004%	4s	10.033	3.146%	17s	PoMtVRS(CaDA)	10.593	0.776%	4s	17.174	1.440%	20s	
	PoMtVRS(CaDA)-EAS	6.552	0.679%	17.2m	9.915	1.929%	1.33h	PoMtVRS(CaDA)-EAS	10.541	0.288%	19.3m	17.047	0.700%	1.42h	
	PoMtVRS(CaDA)-EAS-LoRA	6.566	0.889%	8.4m	9.938	2.169%	55.1m	PoMtVRS(CaDA)-EAS-LoRA	10.550	0.364%	6.2m	17.081	0.897%	46.3m	